PROJECT TECHNICAL DOCUMENTATION

AERO_AUTOMATION

2D 다요소 날개 형상 최적화를 위한 ANSYS Fluent 배치 해석 자동화 도구 — 전체 기술 문서

저장소	luminalaxis/AERO_AUTOMATION
문서화 기준 코드	브랜치 `claude/youthful-wright-aumv9e`, 커밋 `96db077` (2026-10-30)
패키지	`aero-automation` 0.1.0 · Python ≥ 3.11 · 명령어 `aero-auto`
코드 규모	소스 13개 모듈 2,826줄 · 테스트 6개 파일 871줄 · 예제 1개
검증	pytest 43개 전부 통과 (문서 작성 시 직접 실행)
작성일	2026-10-31

이 문서는 저장소의 소스 코드, README, docs/FLUENT_BASE_CASE.md, 예제 설정, 테스트, git 기록을 모두 읽고 정리한 것입니다. 수치(케이스 수, 메쉬 통계, 스크리닝 결과)는 문서 작성 중에 도구를 직접 실행해 얻은 값입니다. Fluent 실행 결과는 ANSYS 라이선스가 없는 환경이라 포함하지 않았습니다.

목차

1. 개요 **4**

1.1 무엇을 하는 도구인가 4

1.2 현재 예제가 다루는 문제 4

1.3 저장소와 브랜치 현황 4

1.4 기술 스택과 실행 환경 4

2. 전체 워크플로 **6**

3. 설치와 빠른 시작 **7**

3.1 설치 7

3.2 예제 실행 순서 7

3.3 자주 쓰는 옵션 7

4. 저장소 구조 **8**

5. 아키텍처와 모듈 **9**

5.1 모듈 의존 관계 9

5.2 핵심 데이터 구조 9

5.3 모듈별 상세 9

6. 설정 파일 레퍼런스 (project.toml) **12**

6.1 섹션 구성 12

6.2 [solver] 12

6.3 [mesh] 12

6.4 [screening] 13

6.5 [results] 13

6.6 케이스 변수 이름 규칙 14

6.7 저널 템플릿 변수 14

7. 형상 정의 **15**

7.1 2단 형상과 슬롯 변수 15

7.2 3단 형상 15

7.3 다운포스 날개 (downforce = true) 15

7.4 지면 효과 (ground_clearance) 16

7.5 형상(메쉬) 이름 규칙 16

8. 메쉬 생성 **17**

8.1 알고리즘 17

8.2 경계 이름 17

8.3 예제 파일럿 메쉬 17

9. CFD 전 스크리닝 **21**

9.1 저충실도 해석 방법 (panel.py) 21
9.2 선별 규칙 22
9.3 병렬 처리와 성능 22
9.4 지면 모드의 한계와 예제 설정 22
9.5 예제 스크리닝 실행 결과 22
9.6 스크리닝 산출물 24
10. Fluent 배치 실행 26
10.1 실행 방식: 베이스 케이스 + 메쉬 교체 26
10.2 예제 저널 템플릿 (journal.jou) 26
10.3 실행 명령 26
10.4 케이스 실행 절차와 상태 26
10.5 Fluent 베이스 케이스 설정 요약 27
11. 결과 처리 29
11.1 리포트 파일 파싱 29
11.2 물체축 → 풍축 변환 29
11.3 단위 폭당 힘 29
11.4 형상 순위 (ranking.csv) 29
11.5 출력 파일 29
12. 예제 프로젝트: flap_optimization 31
12.1 케이스 구성 31
12.2 속도와 레이놀즈수 31
12.3 예제 설정의 주요 선택 31
12.4 파일럿 확인 순서 32
13. 테스트 33
14. 개발 이력 34
15. 한계, 주의사항, 검토 메모 35
15.1 방법론의 한계 (설계상 의도된 것) 35
15.2 코드 검토 중 발견한 사항 35
부록 A. 부록: Fluent 베이스 케이스 작성 절차 36
부록 B. 부록: 용어 37

1 개요

1.1 무엇을 하는 도구인가

AERO_AUTOMATION은 **2D 다요소 날개(메인 + 슬롯 플랩 1~2개)**의 형상을 최적화할 때 생기는 반복 작업을 자동화하는 파이썬 명령줄 도구입니다. 저장소 첫 커밋의 한 줄 설명 그대로, "귀찮은 케이스 분류·해석"을 자동으로 처리하는 것이 목적입니다.

형상 변수(익형, 요소별 코드 길이, 플랩 각도, 갭, 오버랩)와 운용 조건(지상고, 받음각, 속도)을 설정 파일 하나에 적으면, 도구가 다음을 한 번에 처리합니다.

1. **케이스 매트릭스 생성**: 모든 조합을 펼치고, 이름·분류 폴더를 붙이고, 유속·마하수·레이놀즈수 같은 파생 변수를 계산합니다.
2. **CFD 전 스크리닝**: 패널법과 적분 경계층으로 모든 형상을 저충실도로 해석하고, 민감도 그래프를 그리고, 가망 없는 형상 계열을 미리 제외합니다.
3. **메쉬 자동 생성**: 남은 형상마다 gmsh로 경계층 사각형 + 삼각형 혼합 격자를 만들고 Fluent `.msh` 파일로 직접 씁니다.
4. **Fluent 배치 실행**: 케이스마다 저널을 만들어 Fluent를 실행하고, 실패·시간 초과를 감지하고, 중단 후 재실행 시 완료된 케이스를 건너뜁니다.
5. **결과 정리**: 리포트 파일을 읽어 Cl, Cd, L/D, 단위 폭당 힘을 계산하고, 형상 순위와 그래프를 만듭니다.

1.2 현재 예제가 다루는 문제

저장소의 예제(`examples/flap_optimization`)는 **지면 효과를 받는 다운포스용 뒤집힌 날개**입니다. 움직이는 바닥 위 지상고 30~60 mm에서 주행 속도 40~100 km/h로 달리는 조건이고, 형상 순위 기준은 **최대 다운포스**(`objective = "max_cl"`)입니다.

264	25,344	6,336	3,464
형상 (2단 72 + 3단 192)	전체 케이스 (× 지상고 4 × 받음각 6 × 속도 4)	필요한 메쉬 (형상 × 지상고 × 받음각)	스크리닝 후 CFD 케이스 (86% 절감)

1.3 저장소와 브랜치 현황

브랜치	내용
`main`	초기 커밋만 있음: `README.md`(한 줄 설명)와 파이썬용 `.gitignore`
`claude/youthful-wright-aumv9e`	**실제 구현 전부**. 2026-10-29~30에 걸친 16개 커밋, 29개 파일 4,624줄 추가. main에 아직 병합되지 않음
`claude/gracious-ritchie-5n92tn`	이 문서(PDF)를 올리는 브랜치

> **주의** main 브랜치에는 코드가 없습니다. 도구를 쓰려면 `claude/youthful-wright-aumv9e` 브랜치를 체크아웃하거나 main에 병합해야 합니다. 이 문서는 그 브랜치의 최신 커밋 `96db077` 기준입니다.

1.4 기술 스택과 실행 환경

항목	내용
언어	Python 3.11 이상 (표준 라이브러리 `tomllib` 사용)
필수 의존성	`numpy`(수치 계산), `matplotlib`(그래프, Agg 백엔드로 화면 없이 저장), `gmsh ≥ 4.11`(메쉬 생성, pip 패키지에 실행 파일 포함)

항목	내용
개발 의존성	`pytest`
외부 프로그램	ANSYS Fluent (예제는 2026 R1 = v261 기준). `AWP_ROOTxxx` 환경변수로 자동 탐색
운영체제	Windows와 Linux 모두 고려 (Fluent 실행 인자, 프로세스 트리 종료 방식이 OS별로 다름)
설치 명령	`pip install -e .` → 명령어 `aero-auto` 등록 (`python -m aero_automation` 로도 실행 가능)

> **참고** Linux 서버에서 gmsh를 쓰려면 OpenGL 관련 공유 라이브러리(`libGLU.so.1` 등)가 필요합니다. 이 문서를 만들 때 빈 컨테이너에서는 `libglu1-mesa` 를 설치해야 gmsh가 로드됐습니다. Windows나 일반 데스크톱 Linux에서는 보통 이미 있습니다.

2 전체 워크플로

사용자는 `project.toml`과 Fluent 저널 템플릿(`journal.jou`), Fluent 베이스 케이스(`base_2el.cas.h5`, `base_3el.cas.h5`)를 준비하고, 아래 명령을 순서대로 실행합니다. 각 단계는 독립적으로 다시 실행할 수 있고, 이미 끝난 작업은 건너뜁니다.

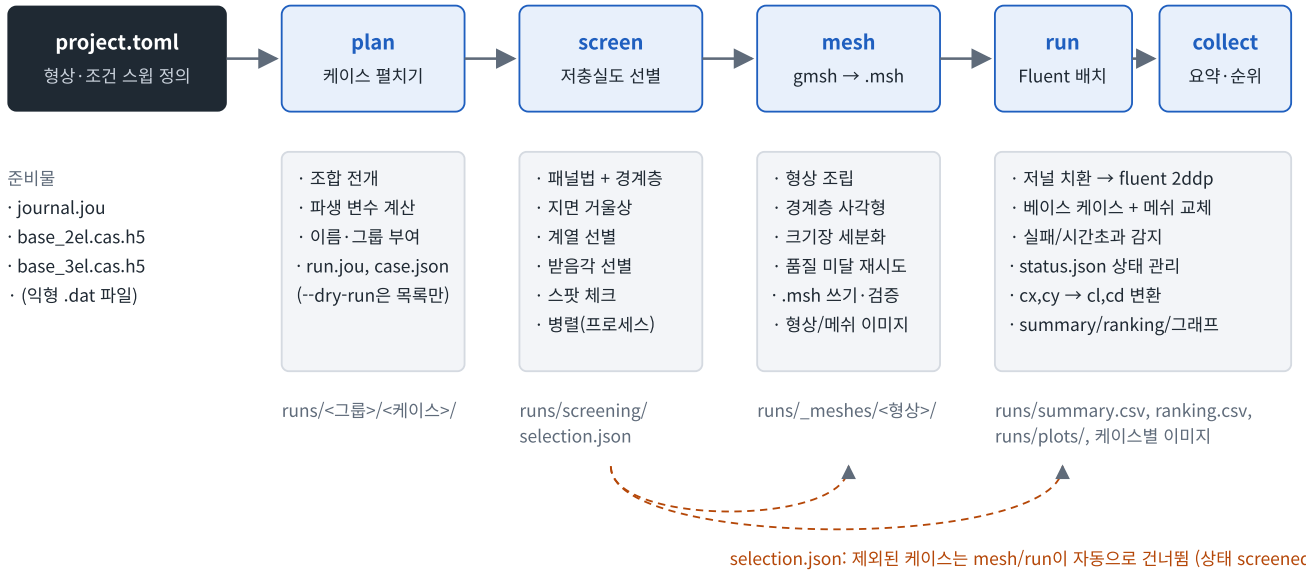


그림 2-1. 명령 단위 파이프라인과 각 단계의 산출물. `run`은 필요한 메쉬가 없으면 `mesh` 단계를 자동으로 수행합니다.

단계별 요약

명령	하는 일	주요 산출물
`plan`	케이스 목록을 표로 출력하고, 케이스 폴더와 치환된 저널(`run.jou`), 변수 기록(`case.json`)을 만듭니다. `--dry-run`이면 출력만 합니다.	케이스 폴더, 실행 명령 예시
`screen`	모든 형상을 패널법 + 경계층 근사로 해석하고, 케이스 세트·속도별로 하위 계열을 제외합니다.	`runs/screening/` CSV, 민감도 그래프, `selection.json`
`mesh`	스크리닝에서 남은 형상의 메쉬를 생성합니다. 설정 지문(fingerprint)이 같은 기존 메쉬는 건너뜁니다.	`runs/_meshes/<형상>/mesh.msh`, 이미지, `mesh_info.json`
`run`	메쉬를 확인·생성한 뒤 남은 케이스를 Fluent로 실행하고, 끝나면 `collect`를 자동 수행합니다.	`solver.log`, `status.json`, 리포트 파일, 결과 케이스
`status`	케이스별 상태(pending, running, done, failed, screened, interrupted)를 표로 보여줍니다.	화면 출력
`collect`	리포트 파일을 다시 읽어 요약·순위·그래프를 만듭니다.	`summary.csv`, `ranking.csv`, `plots/`

3 설치와 빠른 시작

3.1 설치

```
git checkout claude/youthful-wright-aumv9e   # 구현이 있는 브랜치
pip install -e .                             # numpy, matplotlib, gmsh 함께 설치
pip install -e ".[dev]"                      # 테스트까지 하려면 (pytest)
```

3.2 예제 실행 순서

```
cd examples/flap_optimization
aero-auto plan --dry-run   # 케이스 목록 확인 (25,344개, 메쉬 6,336개)
aero-auto screen           # 세트·속도별 저충실도 선별
aero-auto mesh             # 남은 형상만 메쉬 생성 + 이미지
aero-auto run              # 메쉬 확인 → 남은 케이스만 Fluent 실행 → 결과 정리
aero-auto status           # 진행 상태
aero-auto collect          # 결과 다시 정리 (summary, ranking, 그래프)
```

3.3 자주 쓰는 옵션

옵션	의미
`-c, --config 파일`	설정 파일 경로 (기본 `project.toml`). 모든 명령 공통, 서브명령 앞에 씀
`--only 패턴 [패턴 ...]`	케이스 이름 또는 그룹 이름에 대한 와일드카드 필터. 예: `--only "2el_4412c375*"`
`-j, --jobs N`	(run) 동시 실행 케이스 수. 기본은 `[project] max_parallel`. Fluent 라이선스·코어 수 고려
`--force`	(run) 완료된 케이스도 다시 실행 / (mesh) 기존 메쉬도 다시 생성
`--all`	(run, mesh) 스크리닝 결과를 무시하고 전체 대상
`--dry-run`	(plan) 목록만 출력, 파일 생성 안 함

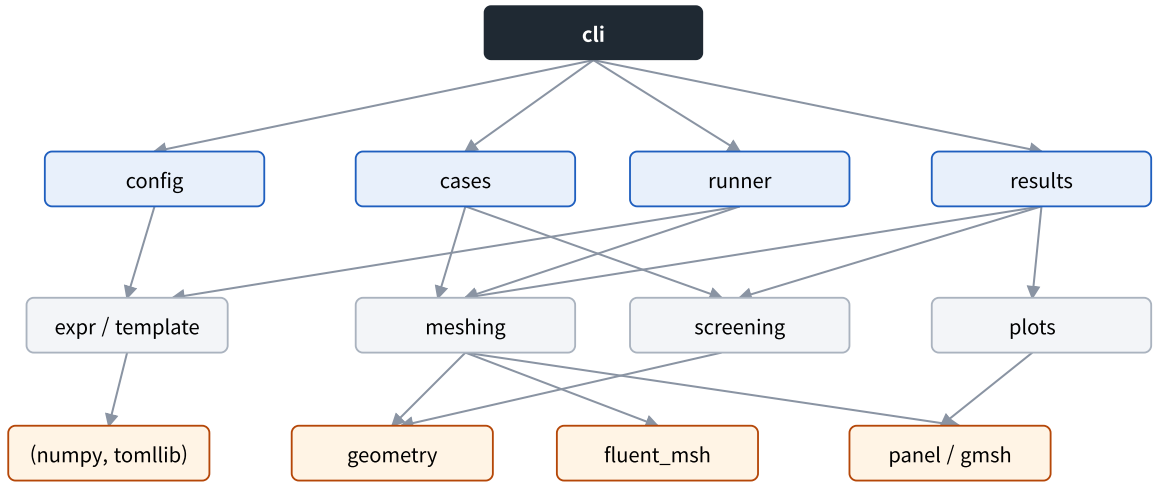
종료 코드: 정상 0, 실행·메쉬 실패가 하나라도 있으면 1, 설정·수식 오류 2, Ctrl+C 중단 130. 중단돼도 다시 `run` 하면 완료된 케이스와 이미 만든 메쉬는 건너뜁니다.

4 저장소 구조

```
AERO_AUTOMATION/
├── README.md                    334줄  사용자 설명서 (한국어)
├── pyproject.toml                      패키지 정의, 명령어 aero-auto 등록
├── .gitignore                          파이썬 표준 + 해석 산출물
├── aero_automation/             패키지 (2,826줄)
│   ├── __init__.py                3   버전 0.1.0
│   ├── __main__.py                6   python -m aero_automation 진입점
│   ├── cli.py                   223   명령줄 인터페이스 (plan/screen/mesh/run/status/collect)
│   ├── config.py                133   project.toml 로딩, 설정 데이터 클래스
│   ├── cases.py                 150   케이스 매트릭스 전개, 이름·그룹, 메쉬 연결
│   ├── expr.py                   70   [derived] 수식의 안전한 계산기 (eval 미사용)
│   ├── template.py               30   저널 템플릿 ${변수} 치환
│   ├── geometry.py              326   NACA/좌표 파일 단면, 플랩 배치, 다운포스 반전, 지면 배치
│   ├── meshing.py               409   gmsh 혼합 격자 생성, 크기장, 품질 재시도
│   ├── fluent_msh.py            179   Fluent ASCII .msh 쓰기/읽기/검증
│   ├── panel.py                 271   Hess-Smith 패널법 + 적분 경계층 (저충실도 해석)
│   ├── screening.py             373   CFD 전 스크리닝: 계열/받음각 선별, 스팟 체크
│   ├── runner.py                279   Fluent 명령 구성, 배치 실행, 상태·실패 관리
│   ├── results.py               243   리포트 파싱, 풍축 변환, 순위, 요약
│   └── plots.py                 306   형상·메쉬·수렴·요약·스크리닝 그래프
├── docs/
│   └── FLUENT_BASE_CASE.md      229   Fluent 베이스 케이스 화면 단위 작성 가이드
├── examples/flap_optimization/
│   ├── project.toml             112   예제 설정 (2단·3단 다운포스 날개, 지면 효과)
│   ├── journal.jou               53   Fluent 저널 템플릿
│   └── sample_meshes.zip      13.7MB   파일럿용 메쉬 2개 (2단, 3단) + 이미지
└── tests/                       (871줄, 43개 테스트)
    ├── fake_solver.py            34   Fluent 대신 리포트 파일을 쓰는 가짜 솔버
    ├── test_workflow.py         118   케이스 전개, 템플릿, 실행·상태·수집, 시간초과
    ├── test_geometry_mesh.py    172   익형, 갭 정확도, .msh 방향 규칙, 전체 파이프라인
    ├── test_multi_element.py    117   3단 형상, 케이스 세트, 세트별 선별
    ├── test_screening.py        228   패널법 이론값, 선별, 속도별 분석, 순위, 힘
    ├── test_downforce.py         90   뒤집힌 날개 거울상, 부호 규칙
    └── test_ground.py           112   지면 배치, 회전, 경계, 지면 효과 경향
```

5 아키텍처와 모듈

5.1 모듈 의존 관계



화살표 = import 방향(주요 관계만). screening·results·meshing은 plots를 필요할 때만(함수 안에서) import해 순환 참조와 시작 시간을 줄입니다.

그림 5-1. 계층 구조: 명령(cli) → 워크플로(cases, runner, results) → 계산 엔진(meshing, screening, plots) → 기초(geometry, fluent_msh, panel)

5.2 핵심 데이터 구조

클래스 (모듈)	역할과 주요 필드
`ProjectConfig` (config)	설정 전체. `template`, `output_dir`, `sweep`, `case_sets`, `constants`, `derived`, `mesh` (dict), `screening` (dict), `solver`, `results`, `max_parallel`. `root` = 설정 파일 폴더
`SolverConfig` (config)	`kind` (fluent/custom), `executable`, `mode`, `processes`, `gui`, `graphics`, `extra_args`, `command`, `timeout_minutes`
`ResultsConfig` (config)	`report_files`, `average_last`, `error_patterns`, `summary_file`, `x_param`, `wind_axes`, `objective`, `split_by`
`Case` (cases)	케이스 하나: `name`, `group`, `params` (모든 변수), `dir` (케이스 폴더), `mesh` (MeshJob)
`MeshJob` (meshing)	메쉬 하나: `name`, `dir`, `geometry_params`, `settings`. `fingerprint()` 로 형상+설정 해시(SHA-1 12자리), `is_current()` 로 재사용 판단
`MeshSettings` (meshing)	`[mesh]` 항목 전체(기본값 포함). `first_cell_height()` 로 y+ 목표 첫 셀 높이 계산
`Element`, `Geometry` (geometry)	요소 좌표(Selig 순서, 메인 코드 = 1 무차원), 앞전 인덱스. `Geometry.info` 에 실제 간격, 지면 위치 등 기록
`InviscidSolution`, `PointResult` (panel)	패널법 해(자유류 x, y 방향 두 해를 저장해 임의 받음각을 선형 조합), 받음각별 Cl, Cd, 요소별 값, 박리 비율
`ScreeningConfig` (screening)	`[screening]` 항목과 기본값
`RunResult` (runner)	실행 결과: `state`, `message`, `elapsed`

5.3 모듈별 상세

config.py — 설정 로딩

- `load_config(path)` : TOML을 읽어 `ProjectConfig` 를 만듭니다. 경로는 설정 파일 폴더 기준으로 절대 경로화합니다.

- 검증: `[project] template` 필수, `solver.kind` 는 fluent/custom, custom이면 `command` 필수, `[cases]` 에 csv·sweep·set 중 하나 필수, 스윕 값은 비어 있지 않은 목록, `[[cases.set]]` 에는 `label/constants/sweep/name/group` 만 허용.
- `safe_name(text)` : 파일 이름에 못 쓰는 문자(`<>:"/\|?*` , 공백)를 `_` 로 바꿉니다.
- 기본 오류 패턴: `Error:` , `Divergence detected` , `floating point exception` .

expr.py — 안전한 수식 계산기

`[derived]` 수식을 `eval` 없이 파이썬 AST로 해석합니다. 허용 요소: 숫자, 정의된 변수, 사칙연산(`+ - * / // % **`), 단항 부호, 함수 `sin cos tan asin acos atan atan2 sqrt exp log log10 radians degrees hypot abs min max round` , 상수 `pi e` . 그 밖의 요소(속성 접근, 키워드 인자 등)는 `ExpressionError` 입니다. 수식은 적힌 순서대로 계산되므로 앞에서 만든 파생 변수를 뒤 수식에서 쓸 수 있습니다(예제: `chord` → `velocity` → `mach` , `reynolds`).

template.py — 저널 치환

`${변수}` 또는 `${변수:포맷}` (예: `${aoa:.2f}` , 파이썬 format 규격)만 치환합니다. Fluent Scheme 문법과 충돌하지 않도록 중괄호 형태만 인식하고, 정의되지 않은 변수가 있으면 목록을 모아 `ConfigError` 를 냅니다.

cases.py — 케이스 매트릭스

- `build_cases(cfg)` : (CSV 행) × (공통 sweep 조합) × (세트별 sweep 조합)을 세트마다 펼칩니다. 값 우선순위는 **`[constants]` < 세트 constants < CSV 행 < 공통 sweep < 세트 sweep**입니다. 세트의 `label` 은 변수 `case_set` 으로 들어갑니다.
- 이름: 변수 `name` 이 있으면 그 값, 없으면 이름 패턴(`str.format`), 패턴도 없으면 `case_001` . 이름이 중복되면 오류. 그룹 패턴이 있으면 케이스 폴더가 `runs/<그룹>/<이름>` .
- `_set_flow_angle` : Fluent 유입 각도 `flow_angle` 을 정합니다. 지면이 있으면 0(받음각은 형상 회전), 다운포스 날개는 `-aoa` , 그 외 `aoa` .
- `_attach_meshes` : 같은 형상의 케이스는 메쉬 하나를 공유합니다. 형상 안에서 `reynolds` 가 다르면(속도 스윕) **가장 큰 Re**로 첫 셀 높이를 잡아 모든 속도에서 y+가 목표 이하가 되게 합니다.
- `filter_cases` : `--only` 패턴을 케이스 이름과 그룹 이름 양쪽에 `fnmatch` 로 적용합니다.

geometry.py — 형상 생성

- `naca4(code)` : NACA 4자리 식(코사인 분포 160점, 기본은 뭉툭한 뒷전 계수 −0.1015). `read_airfoil_file` : Selig 또는 Lednicer 형식 `.dat` (첫 줄이 점 개수면 Lednicer로 판단, 쉼표 구분도 허용).
- `normalize` : 앞전(뒷전 중점에서 가장 먼 점)을 원점에, 코드를 1로, 코드선을 x축에 맞추고 Selig 순서(뒷전 윗면 → 앞전 → 뒷전 아랫면)로 통일, 중복 점 제거.
- `_place_flap` : 플랩을 크기 조정·회전한 뒤 x 위치를 overlap으로 정하고, **세로 위치를 이분법(60회)으로 찾아** 앞 요소 뒷전(아랫면 끝점)에서 플랩 표면까지 부호 거리가 정확히 gap이 되게 합니다. 해가 없으면 "overlap을 줄이거나 gap을 늘리라"는 오류, 앞선 모든 요소와 겹치면 오류.
- `build_geometry` : 플랩 배치 결과를 형상 변수별로 캐시(최대 512개)하고, 지면 배치(받음각 회전 + 지면 위치)만 매번 새로 계산합니다. 지면 모드 스크리닝에서 받음각마다 반복 호출되기 때문입니다.
- `mirror` : 다운포스 날개용 위아래 반전. 점 순서를 뒤집어 Selig 순서를 유지합니다.
- `_place_over_ground` : 메인 앞전을 중심으로 받음각만큼 회전하고, `ground_ref` 에 따라 지면 높이 `ground_y` 를 정합니다. `main_le` 기준에서 날개가 지면을 뚫으면 형상 불가.

meshing.py — 메쉬 생성

gmsh로 2D 혼합 격자를 만들고 `fluent_msh.write_msh` 로 Fluent 형식을 직접 씁니다. 상세 알고리즘은 8장에 정리했습니다. `mesh_name(params)` 는 형상을 사람이 읽을 수 있는 이름으로 만드는 함수로, 메쉬 폴더 이름과 스크리닝·순위의 형상 식별자로 쓰입니다(7.5절).

fluent_msh.py — Fluent 메쉬 파일

- `build_faces`: 셀을 반시계로 정렬한 뒤 변마다 왼쪽 셀을 c0, 오른쪽 셀을 c1로 기록합니다(Fluent 2D 규칙: 면 n0→n1 진행 방향 왼쪽이 c0). 경계면은 c1 = 0이고 c0가 유체 셀입니다.
- `write_msh`: 헤더, 차원 `(2 2)`, 노드(섹션 10), 셀(섹션 12, 삼각형 1/사각형 3 혼합이면 셀별 타입 목록), 면(섹션 13, 존별), 존 이름(섹션 45)을 16진수 인덱스로 씁니다. 경계 지정이 빠진 면, 중복 지정된 면이 있으면 오류.
- `check_msh`: 쓴 파일을 다시 읽어 Fluent 규칙대로 셀 면적을 계산하고, 면적이 0 이하인 셀(`bad_cells`)과 닫히지 않은 셀(`open_cells`)을 셉니다. 하나라도 있으면 메쉬 생성 실패로 처리합니다.
- 경계 조건 타입 코드: interior 2, wall 3, pressure-inlet 4, pressure-outlet 5, symmetry 7, pressure-far-field 9, velocity-inlet 10.

panel.py — 저충실도 공력 해석

Hess-Smith 패널법(비점성), Thwaites–Michel–Head 적분 경계층(점성 근사), Squire-Young 항력, Prandtl-Glauert 압축성 보정, 지면 거울상. 9.1절에 정리했습니다.

screening.py — CFD 전 스크리닝

형상별 저충실도 해석을 병렬로 실행하고, 계열 선별·받음각 선별·스팟 체크로 케이스를 고릅니다. 9장에 정리했습니다.

runner.py — 배치 실행

Fluent 실행 파일 탐색, 명령 구성, 케이스별 실행과 상태 관리, 실패 판정, 병렬 실행을 담당합니다. 10장에 정리했습니다.

results.py — 결과 처리

Fluent 리포트 파일 파싱, 물체축 → 풍축 변환, 단위 폭당 힘, 형상 순위, 그래프 생성, 스크리닝 대비 CFD 비교를 담당합니다. 11장에 정리했습니다.

plots.py — 이미지

함수	출력
`plot_geometry`	`geometry.png`: 요소별 윤곽, 지면, 제목에 받음각·지상고·최하단 높이·앞전 높이·플랩별 실제 최소 간격
`plot_mesh`	`mesh.png` (근거리 전체), `mesh_slot1/2.png` (슬롯 확대, 다운포스면 위쪽), `mesh_le.png` (메인 앞전). 보이는 영역의 셀만 그려 속도 확보
`plot_convergence`	케이스별 `convergence.png`: 리포트 값별 수렴 이력, y축은 마지막 20% 구간 기준
`plot_summary`	`cl_alpha.png`, `cd_alpha.png`, `ld_alpha.png`, `polar.png`. 형상 12개 초과면 범례 생략
`plot_condition_trend`	`by_<조건>.png`: 속도 등 조건에 따른 상위 10개 형상의 최대 L/D(또는 Cl)
`plot_screening`	`sensitivity_main_effects.png`, `sensitivity_importance.png`, `screening_polars.png`, `screening_scatter.png`
`plot_screening_vs_cfd`	`screening_vs_cfd_cl.png`, `screening_vs_cfd_ld.png`: 추정값 vs CFD 산점도와 상관계수

6 설정 파일 레퍼런스 (project.toml)

설정 파일 하나가 프로젝트 전체를 정의합니다. 경로는 모두 설정 파일이 있는 폴더 기준입니다.

6.1 섹션 구성

섹션	내용
`[project]`	`template` (필수, 저널 템플릿), `name` (기본: 폴더 이름), `output_dir` (기본 `runs`), `max_parallel` (기본 1)
`[solver]`	솔버 실행 설정 (6.2절)
`[constants]`	모든 케이스 공통 값
`[cases]`	`csv` (행 단위 케이스 파일, 선택), `name` (케이스 이름 패턴), `group` (분류 폴더 패턴)
`[cases.sweep]`	전체 조합 변수. 세트가 있으면 모든 세트에 공통 적용
`[[cases.set]]`	케이스 세트(예: 2단, 3단): `label`, `constants`, `sweep`, `name`, `group`
`[derived]`	파생 변수 수식 (예: `velocity = "speed_kmh / 3.6"`)
`[mesh]`	메쉬 설정 (6.3절). 섹션이 없으면 메쉬 생성 없이 동작
`[screening]`	스크리닝 설정 (6.4절)
`[results]`	결과 처리 설정 (6.5절)

6.2 [solver]

항목	기본값	내용
`kind`	`"fluent"`	`"fluent"` 또는 `"custom"` (임의 명령, 테스트에서 가짜 솔버에 사용)
`executable`	비움	Fluent 실행 파일. 비우면 `AWP_ROOTxxx` 환경변수 중 가장 높은 버전 → PATH의 `fluent` 순으로 탐색
`mode`	`"3ddp"`	2d, 2ddp, 3d, 3ddp. **2D 메쉬이므로 `"2ddp"`를 명시해야 함**(예제는 명시)
`processes`	4	`-t` 인자 (병렬 프로세스 수)
`gui`	false	true면 GUI 표시
`graphics`	false	true면 GUI 없이 그래픽만 살려 이미지 저장 가능 (Windows `-hidden`, Linux `-gu -driver null`). false면 `-g`
`timeout_minutes`	없음	초과 시 프로세스 트리 강제 종료, failed 처리
`extra_args`	[]	명령 끝에 붙일 추가 인자
`command`	없음	custom일 때 명령 템플릿 목록. 케이스 변수 + `${exe}`, `${journal}`, `${python}` 사용 가능

6.3 [mesh]

길이는 모두 메인 코드 c 기준 배수입니다. `[mesh]` 항목과 같은 이름의 케이스 변수가 있으면 케이스 값이 우선합니다(예: `chord`, `reynolds`). 알 수 없는 항목은 오류입니다.

항목	기본값	내용
`chord`	1.0	메인 코드 [m]. 메쉬 좌표 스케일
`reynolds`, `y_plus`	3e6, 1.0	첫 셀 높이 계산용 (평판 난류 마찰계수 추정)
`growth_rate`	1.15	경계층 성장률

항목	기본값	내용
`bl_thickness`	0.02	경계층 사각형 영역 최대 두께
`gap_bl_fraction`	0.3	슬롯·지면 통로에서 경계층 두께 ≤ 최소 간격 × 이 값
`surface_cells`, `flap_surface_cells`	200, 150	메인 / 플랩마다 윗면·아랫면 각각의 표면 셀 수
`te_cells`	6	뭉툭한 뒷전 면의 셀 수
`surface_bump`	0.15	앞전·뒷전 셀 집중도 (작을수록 양 끝에 집중)
`near_size`, `wake_size`	0.01, 0.01	물체 주변 / 후류 셀 크기
`wake_length`, `wake_angles`	3.0, [−10, 0, 10, 20]	후류 세분화 길이와 방향 [deg]
`farfield_radius`, `farfield_size`	50, 5.0	(지면 없음) 원형 원거리 경계 반지름과 셀 크기
`ground_upstream`, `ground_downstream`, `ground_height`	5, 12, 6	(지면 있음) 메인 앞전~inlet, 메인 앞전~outlet, 지면~위쪽 경계
`ground_far_size`	0.5	(지면 있음) 바깥 경계 근처 셀 크기
`min_quality`	0.05	최소 셀 품질 (삼각형 SICN, 사각형 scaled Jacobian)

6.4 [screening]

항목	기본값	내용
`protect`	`["*gap", "*overlap", "main_chord", "chord", "ground_clearance"]`	이 변수 값만으로는 제외하지 않음 (와일드카드 가능)
`keep_top`	3	항상 유지할 상위 계열 수
`keep_within`	0.05	최고 계열 점수 대비 이 비율 이내면 유지
`spot_check`	1	제외된 계열 중 대표 케이스만 CFD로 검증할 계열 수
`prune_aoa`, `aoa_margin`	true, 2.0	실속 이후 받음각 제외 여부, 여유각 [deg]
`sep_limit`	0.10	실속 판정: 메인 흡입면 박리 구간 비율
`aoa_step`	1.0	실속각 탐색 간격 [deg]
`transition`	`"turbulent"`	`"free"` 면 Michel 기준 자연 천이
`reynolds`, `mach`	비움	비우면 케이스 변수 → `[mesh] reynolds` → 3e6 (마하는 케이스 변수 → 0)
`metric`	비움	`max_ld` / `max_cl`. 비우면 `[results] objective`
`n_side`	80	물체당 윗면·아랫면 패널 수
`workers`	0	병렬 프로세스 수 (0 = CPU 코어 수)

> **참고** README의 표는 `protect` 기본값을 `["*gap", "*overlap", "main_chord", "chord"]` 로 적고 있지만, 코드(`ScreeningConfig`)의 실제 기본값에는 `"ground_clearance"` 도 들어 있습니다. 위 표는 코드 기준입니다.

6.5 [results]

항목	기본값	내용
`report_files`	`["*.out"]`	케이스 폴더에서 읽을 리포트 파일 패턴
`average_last`	1	마지막 N개 반복의 평균을 결과값으로 사용 (예제 100)

항목	기본값	내용
`error_patterns`	3개 (5.3절)	로그에서 찾으면 실패로 판정할 문자열 (대소문자 무시)
`summary_file`	`"summary.csv"`	요약 파일 이름
`x_param`	`"aoa"`	받음각 변수 이름 (그래프 x축)
`wind_axes`	true	cx, cy 리포트를 cl, cd로 변환
`objective`	`"max_ld"`	형상 순위 기준: `max_ld` 또는 `max_cl`
`split_by`	[]	조건별로 나눠 순위·그래프·스크리닝 (예: `["speed_kmh"]`)

6.6 케이스 변수 이름 규칙

도구는 다음 변수 이름에 특별한 의미를 둡니다. 나머지 변수는 저널 치환과 결과 표에만 쓰입니다.

변수	의미
`main_airfoil`, `main_chord`	메인 단면(NACA 4자리 또는 좌표 파일 경로), 메인 코드 길이 [mm 등 입력 단위]
`flapN_airfoil/chord/deflection/gap/overlap`	N번째 플랩 (N = 1~3). `flapN_airfoil`이 있는 데까지 요소 수가 정해짐
`chord`	메인 코드 [m]. 메쉬 스케일, Re, 계수 기준 길이, 단위 폭당 힘
`downforce`	true면 뒤집힌 날개
`ground_clearance`, `ground_ref`	지상고와 기준 (`lowest` / `main_le`)
`aoa` (= `x_param`)	받음각
`flow_angle`	자동 계산되는 Fluent 유입 각도 (직접 지정하면 그 값 유지)
`reynolds`, `mach`, `velocity`, `density`	메쉬 첫 셀 높이, 스크리닝 조건, 단위 폭당 힘 계산에 사용 (`density` 없으면 1.225)
`case_set`	세트의 `label` (자동)
`name`	있으면 케이스 이름으로 직접 사용

6.7 저널 템플릿 변수

모든 케이스 변수에 더해 다음 변수를 쓸 수 있습니다. 경로는 Windows에서도 슬래시(`/`) 형태로 들어갑니다(역슬래시의 이스케이프 문제 방지).

변수	값
`${case_name}`, `${group}`	케이스 이름, 그룹 이름
`${case_dir}`, `${project_dir}`	케이스 폴더, 설정 파일 폴더 (절대 경로)
`${nprocs}`	`[solver] processes`
`${mesh_file}`, `${mesh_name}`	이 케이스의 메쉬 파일 경로, 메쉬 이름 (`[mesh]`가 있을 때)

7 형상 정의

7.1 2단 형상과 슬롯 변수

길이(코드, 갭, 오버랩)는 모두 **같은 단위(예제는 mm)**로 받고, 내부에서는 메인 코드 = 1로 무차원화합니다. `main_chord`가 없으면 메인 코드 = 1로 보고 나머지를 비율로 해석합니다.

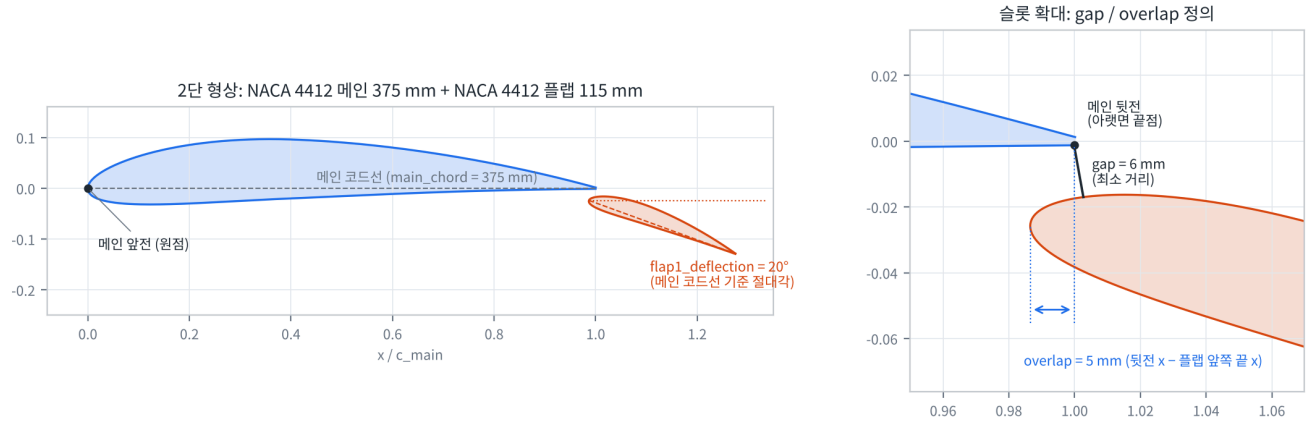


그림 7-1. 도구의 `build_geometry`로 실제 계산한 2단 형상. 갭은 메인 뒷전(아랫면 끝점)에서 플랩 표면까지의 최소 거리, 오버랩은 메인 뒷전 x − 플랩 앞쪽 끝 x(+면 플랩이 메인 아래로 들어감)

변수	정의
`flapN_deflection`	메인 코드선 기준 **절대각** [deg], 뒷전이 내려가는 방향이 +. 3단에서 flap2 각도도 메인 기준
`flapN_gap`	바로 앞 요소(메인 또는 flap{N−1}) 뒷전 아랫면 끝점 ~ 플랩 표면 최소 거리. 0보다 커야 함
`flapN_overlap`	앞 요소 뒷전 x − 플랩 앞쪽 끝 x
`flapN_chord`	플랩 코드. 메인 코드의 0~1.5배 범위가 아니면 오류

도구는 주어진 overlap에서 gap이 정확히 맞도록 플랩 높이를 자동 계산합니다(이분법). 불가능한 조합이나 요소끼리 겹치는 조합은 해당 형상만 실패 처리하고, 앞선 **모든** 요소와의 실제 최소 간격은 `mesh_info.json`의 `flaps[].clearance`에 기록합니다. 테스트(`test_flap_gap_is_exact`)가 갭 정확도를 검증합니다.

7.2 3단 형상

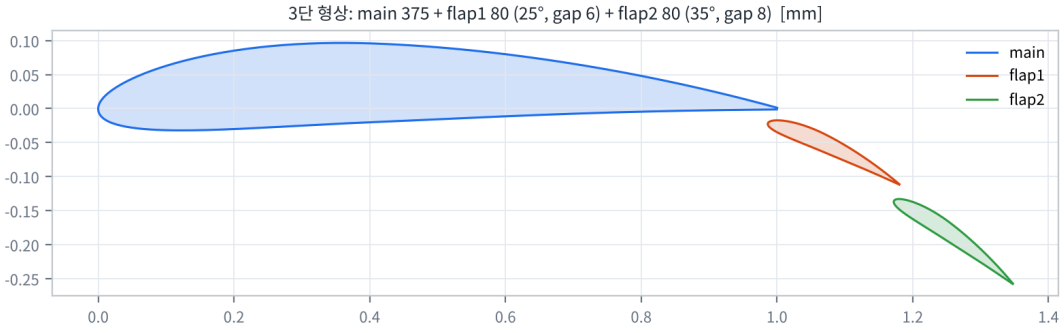


그림 7-2. 3단 형상 (예제의 3단 파일럿 형상과 같은 변수). flap2의 갭·오버랩은 flap1 뒷전 기준

7.3 다운포스 날개 (downforce = true)

- 전체 형상(메인 + 플랩)을 위아래로 뒤집습니다. 플랩은 앞 요소 뒷전의 **위쪽**에 놓이고, 갭·오버랩·각도 정의는 거울상 그대로입니다. 메쉬 이름 끝에 `_inv`가 붙습니다.

- **받음각 `aoa`는 다운포스가 커지는 방향이 +**입니다. 지면이 없으면 Fluent 유입 각도 `flow_angle = −aoa`가 자동 계산됩니다.
- 결과의 `cl`은 다운포스 계수(아래 방향 +), `l/d`는 다운포스/항력, 힘은 `downforce_N_per_m`입니다. 요소별(`cl-main`, `cl-flap1` 등)도 같은 부호입니다.
- 자유류에서는 뒤집힌 날개가 똑바른 날개의 정확한 거울상이라, 스크리닝은 똑바른 형상의 양력을 다운포스로 씁니다 (테스트 `test_screening_downforce_equals_upright`).

7.4 지면 효과 (ground_clearance)

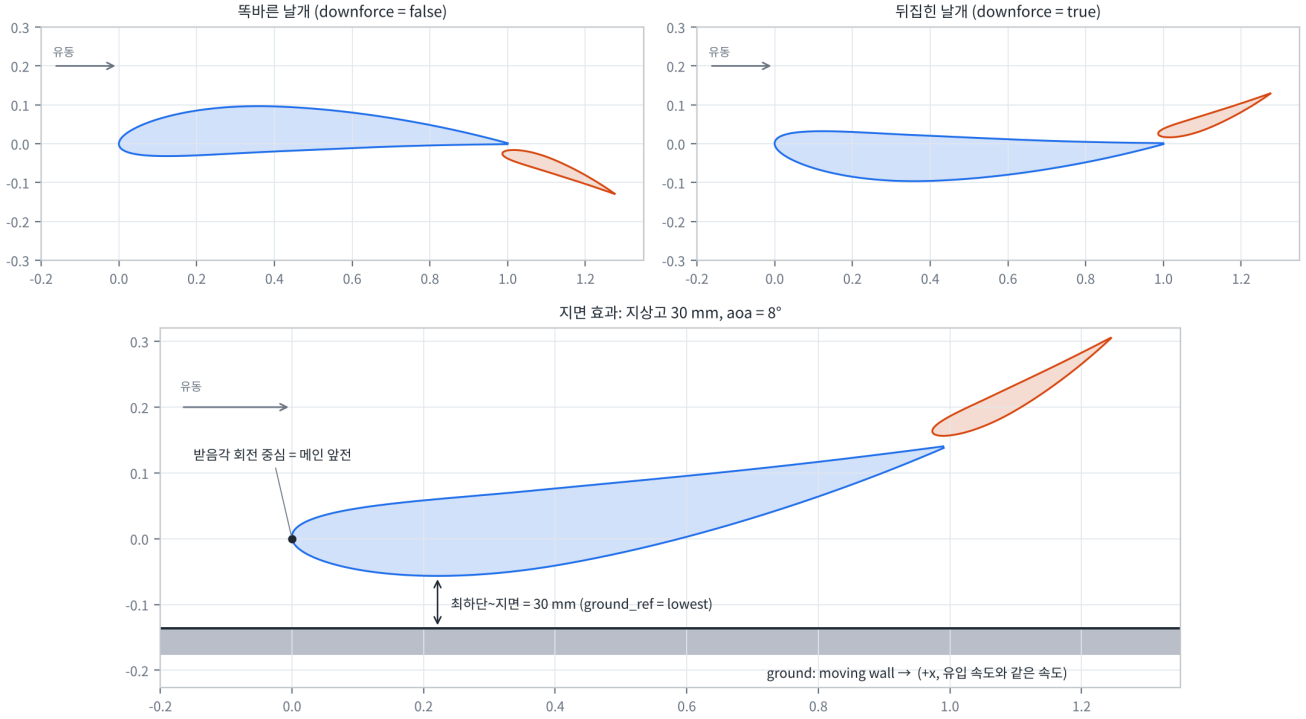


그림 7-3. 같은 2단 형상의 똑바른 날개, 뒤집힌 날개, 그리고 지상고 30 mm·받음각 8°에서 지면 위에 배치한 모습 (도구로 계산)

- **해석 영역**이 직사각형으로 바뀝니다: 앞 `inlet`(velocity-inlet), 뒤 `outlet`(pressure-outlet), 위 `top`(symmetry), 아래 `ground`(wall → Fluent에서 moving wall).
- **받음각**은 메인 앞전을 중심으로 날개 전체를 실제로 회전시켜 반영합니다. 유동은 항상 지면과 평행해서 `flow_angle = 0`입니다. 그래서 **받음각마다 메쉬가 따로** 생기고, 속도만 다른 케이스끼리 메쉬를 공유합니다. 메쉬 이름 끝에 `_h<지상고>_a<받음각>`이 붙습니다.
- **지상고 기준**: `lowest`(기본) = 회전한 날개의 최하단~지면, `main_le` = 메인 앞전~지면. 뒤집힌 날개는 아랫면이 앞전보다 코드의 약 10% 아래로 내려오므로 `main_le` 기준에서는 낮은 지상고에서 날개가 지면을 뚫는 케이스가 생기고, 그 케이스는 형상 불가로 처리됩니다.

7.5 형상(메쉬) 이름 규칙

```
<메인 익형>c<메인 코드> [_<플랩 익형>c<코드>d<각도>g<갭>o<오버랩>] × 플랩 수 [_inv] [_h<지상고>_a<받음각>]
```

예: `4412c375_4412c115d20g6o5_inv_h30_a4` = NACA 4412 메인 375 mm + NACA 4412 플랩 115 mm(20°, 갭 6, 오버랩 5), 뒤집힌 날개, 지상고 30 mm, 받음각 4°. 좌표 파일이면 파일 이름(확장자 제외)이 들어갑니다. 결과 처리에서는 받음각을 뺀 이름을 `geometry` 열로 써서 형상 단위 비교를 합니다.

8 메쉬 생성

8.1 알고리즘

1. **형상 조립**: `build_geometry`로 요소 좌표를 만듭니다(메인 코드 = 1).
2. **경계층 두께 한도**: `bl_thickness`를 기본으로, 다요소면 `gap_bl_fraction × 슬롯 최소 간격`, 지면이 있으면 `gap_bl_fraction × 날개 최하단~지면 거리`까지 줄입니다. 양쪽 벽에서 자라는 경계층이 슬롯에서 만나지 않게 하기 위함입니다.
3. **표면**: 각 요소의 윗면·아랫면을 스플라인으로 만들고 Bump 분포(`surface_bump`)로 표면 셀을 앞전·뒷전에 집중합니다. 뾰족한 뒷전은 부채꼴(fan) 점, 뭉툭한 뒷전은 `te_cells`개 셀의 직선 면.
4. **바깥 경계**: 지면 없으면 반지름 50c 원(`farfield`), 지면 있으면 직사각형(inlet/outlet/top/ground).
5. **경계층**: gmsh `BoundaryLayer` 필드(사각형, 첫 셀 높이 = y+ 목표, 성장률 1.15).
6. **크기장**(최솟값 합성): 물체 거리 기반(`near_size`), 후류선(각 뒷전에서 `wake_angles` 방향, 다운포스면 각도 반전, 지면 아래로 내려가는 선은 자름), 뒷전 주변(경계층 두께/5), 슬롯(그 슬롯 최소 간격/6), 지면 통로(지상고/8).
7. **생성**: Frontal-Delaunay(알고리즘 6). 품질을 삼각형은 SICN, 사각형은 scaled Jacobian으로 평가합니다(경계층 사각형은 종횡비가 커서 SICN이 낮게 나오는 게 정상이기 때문).
8. **재시도**: gmsh 오류나 품질 미달이면 경계층 두께를 1.0 → 0.6 → 0.35배로 줄여 최대 3회 시도합니다.
9. **쓰기·검증**: 좌표에 `chord`[m]를 곱해 Fluent `.msh`로 쓰고, 다시 읽어 셀 면적·닫힘을 검증합니다. 실패하면 `mesh_error.txt`를 남기고 해당 형상의 케이스를 failed 처리합니다.
10. **기록**: `mesh_info.json`(지문, 형상, 설정, 첫 셀 높이, 경계층 층수, 셀 수, 최소 품질, 시도 횟수, 존별 면 수)과 이미지 4종.

```
첫 셀 높이 (메인 코드 기준): Cf = 0.026 / Re^(1/7), y₁ = y⁺ · √(2 / Cf) / Re 경계층 층수: n = ⌊ ln(1 + t·(r − 1) / y₁) / ln r ⌋ (t = 경계층 두께, r = 성장률)
```

재사용 규칙: 메쉬 폴더에 `mesh.msh`와 `mesh_info.json`이 있고, 형상 변수 + 메쉬 설정의 SHA-1 지문이 같으면 다시 만들지 않습니다. 설정을 바꾸면 자동으로 재생성 대상이 됩니다. 재생성 전에 기존 파일을 먼저 지워서, 재생성이 실패했을 때 옛 메쉬가 쓰이지 않게 합니다.

8.2 경계 이름

경계	타입	비고
`main`, `flap1`, `flap2`	wall	요소별 벽면. 요소별 힘 리포트에 사용
`ground`	wall	지면 모드. Fluent에서 moving wall로 설정. 지면 경계층은 따로 두지 않음
`inlet` / `outlet` / `top`	velocity-inlet / pressure-outlet / symmetry	지면 모드
`farfield`	pressure-far-field	자유류 모드
`interior-fluid`, 셀 영역 `fluid`	interior / fluid	

같은 구성(2단 또는 3단) 안에서는 경계 이름이 같아서, Fluent에서 **메쉬만 교체해도 경계조건·리포트 설정이 유지**됩니다. 이것이 베이스 케이스 + 메쉬 교체 방식의 전제입니다.

8.3 예제 파일럿 메쉬

`sample_meshes.zip`에 들어 있는 두 메쉬의 `mesh_info.json` 값입니다. 둘 다 Re 713,111(100 km/h, 메인 375 mm)로 첫 셀 높이를 잡았습니다.

	2단 (main + flap1)	3단 (main + flap1 + flap2)
메쉬 이름	`4412c375_4412c115d20g6o5_inv_h30_a4`	`4412c375_4412c80d25g6o5_4412c80d35g8o3_inv_h30_a4`
셀 수 (삼각형 / 사각형)	69,557 (53,641 / 15,916)	92,027 (69,253 / 22,774)
노드 수	43,345	58,175
첫 셀 높이	12.1 μm	12.1 μm
경계층 두께 / 층수	0.0048c (1.8 mm) / 22층	0.0048c / 22층
최소 셀 품질	0.320	0.326
생성 시도	1회	1회
지상고 (최하단 / 메인 앞전)	30.0 mm / 57.8 mm	30.0 mm / 57.8 mm
슬롯 최소 간격	flap1 6.0 mm	flap1 6.0 mm, flap2 8.0 mm

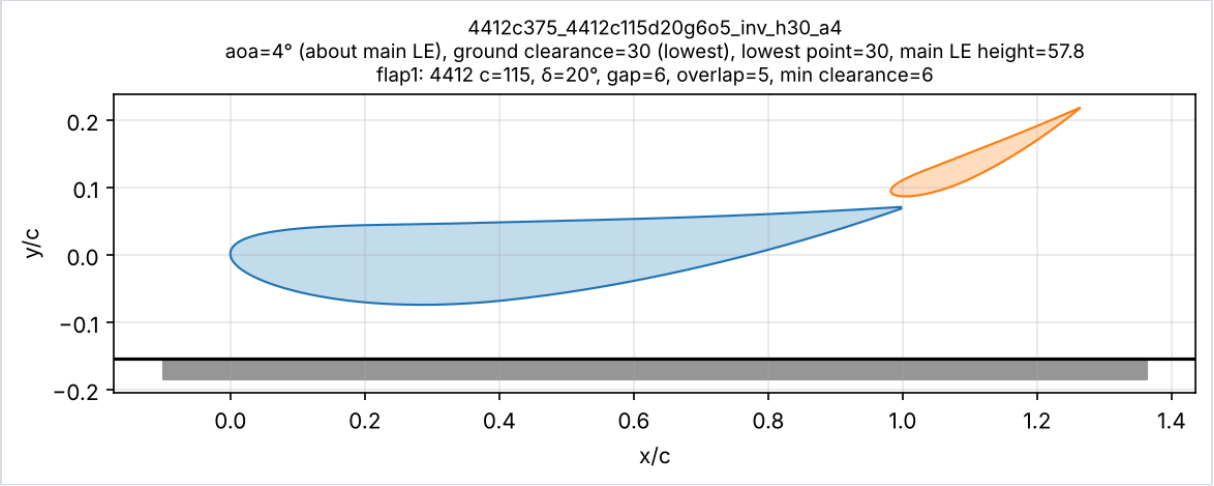


그림 8-1. 도구가 자동 저장하는 `geometry.png` (2단 파일럿 형상)

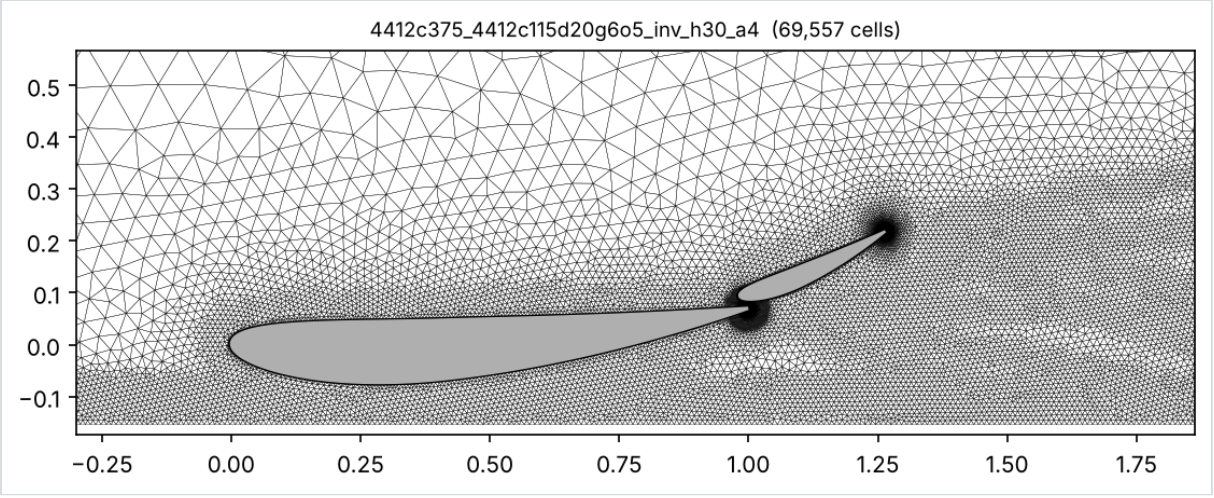


그림 8-2. `mesh.png` : 날개~지면 통로 세분화와 후류 세분화가 보임 (2단)

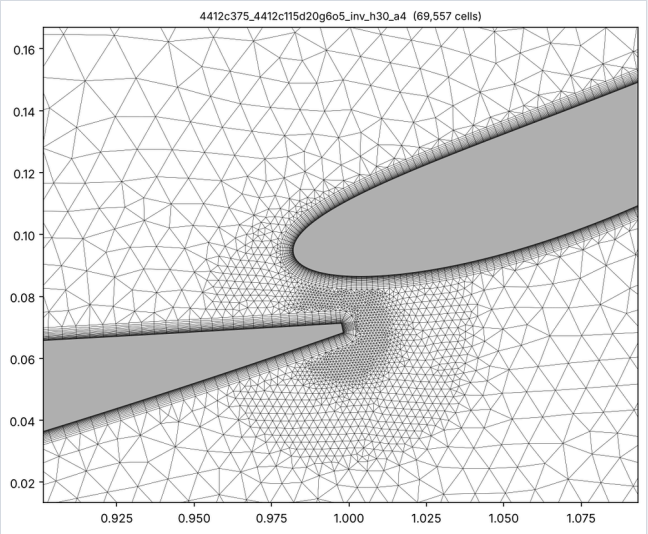


그림 8-3. `mesh_slot1.png`: 슬롯 경계층 (2단)

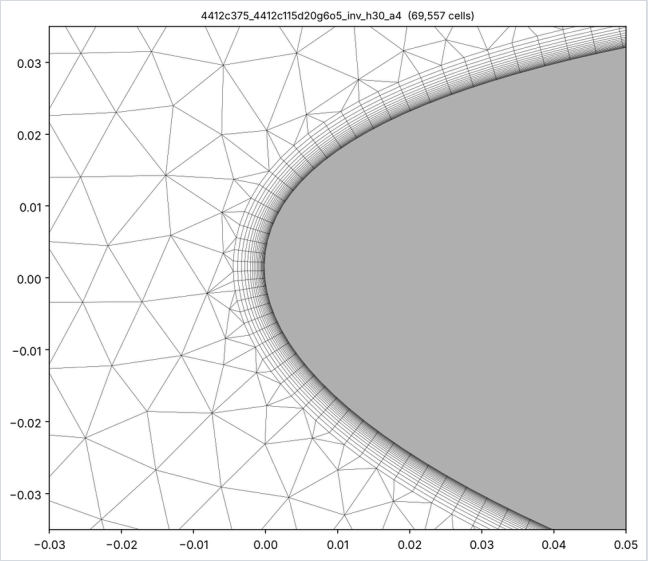


그림 8-4. `mesh_le.png`: 메인 앞전 경계층 (2단)

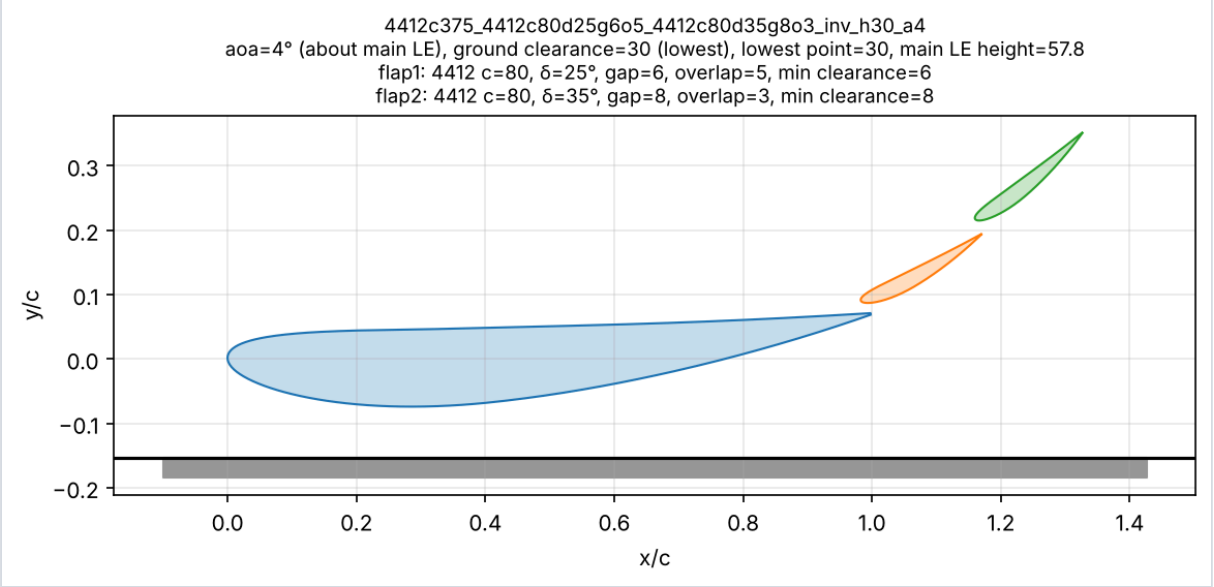


그림 8-5. 3단 파일럿 형상의 `geometry.png`

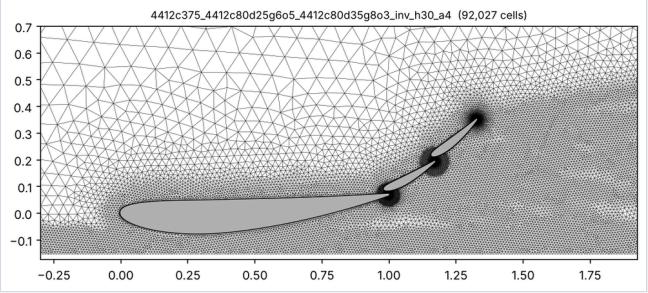


그림 8-6. 3단 `mesh.png`

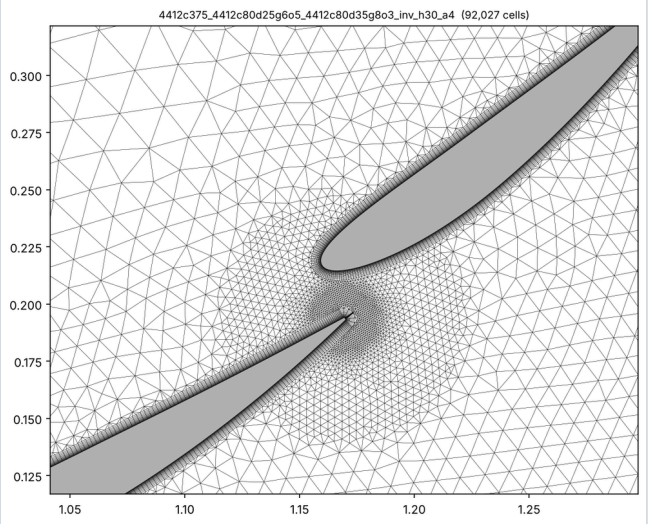


그림 8-7. 3단 `mesh_slot2.png` (flap1↔flap2 슬롯)

메쉬 독립성은 기준 형상 하나로 `surface_cells`, `near_size` 등을 2~3단계 바꿔 Cl/Cd 수렴을 확인한 뒤, 최적화 동안에는 설정을 고정하는 것이 README의 권장 사항입니다. 설정이 같아야 형상 간 비교에서 메쉬 차이로 생기는 노이즈가 줄어듭니다.

9 CFD 전 스크리닝

익형·코드·각도 조합을 여러 개 넣으면 케이스가 금방 수만 개가 됩니다. `screen`은 Fluent를 돌리기 전에 모든 형상을 저충실도로 해석해 민감도를 보여주고, 가망 없는 케이스를 미리 제외합니다.

9.1 저충실도 해석 방법 (panel.py)

비점성: Hess-Smith 패널법

- 각 요소를 윗면·아랫면 80개씩(코사인 분포, 양 끝 집중) 직선 패널로 나눕니다.
- 패널마다 일정 세기 소스, 물체마다 일정 세기 와류를 둡니다. 미지수 = 패널 수 + 물체 수.
- 조건: 모든 패널 중점에서 법선 속도 0, 물체마다 Kutta 조건(첫·마지막 패널 접선 속도 합 = 0). 요소 개수와 무관하게 같은 방식으로 풀립니다.
- 영향 계수는 복소수 해석식 `ln(z/(z−L))`으로 계산하고, 자기 패널 중점은 반올림 오차로 로그 분기가 뒤집히지 않도록 해석값 −iπ로 고정합니다.
- 자유류 (1, 0)과 (0, 1) 두 경우를 풀어 저장하고, 임의 받음각은 두 해의 선형 조합으로 얻습니다(같은 형상의 해를 속도가 달라도 재사용).
- **지면 효과**: 지면 `y = ground_y`에 대한 거울상 물체(소스 같은 세기, 와류 반대 세기)를 더해 지면 법선 속도를 0으로 만듭니다. 지면 경계층이 없는 움직이는 바닥과 같은 조건입니다.
- 압축성: Prandtl-Glauert 보정 `Cp / √(1 − M²)`.

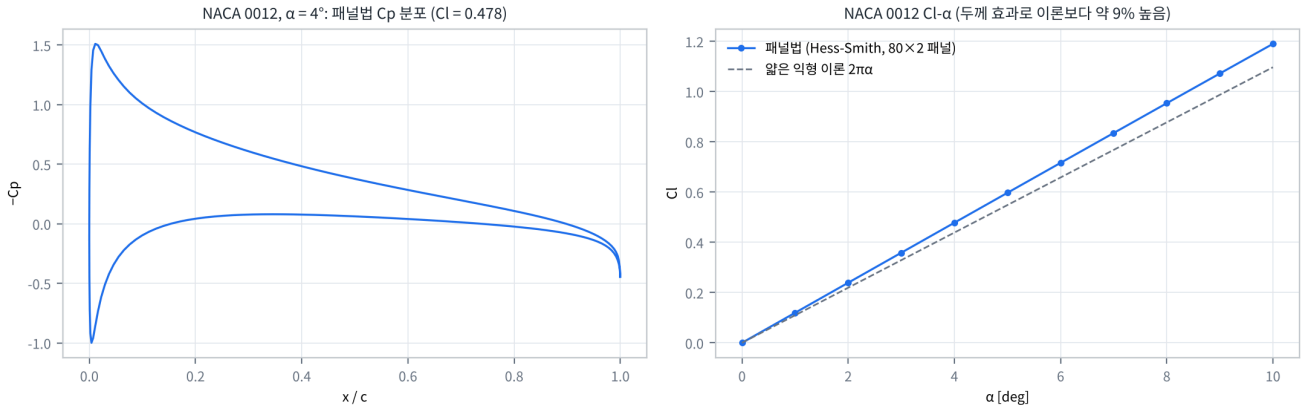


그림 9-1. 패널법 검증 (도구로 계산). NACA 0012 α = 4°에서 Cl = 0.478로, 테스트 `test_panel_matches_theory`가 이 값을 확인합니다.

점성 근사: 적분 경계층

1. 정체점을 접선 속도 부호가 바뀌는 곳에서 선형 보간으로 찾고, 윗면·아랫면을 따라 호길이를 누적합니다.
2. **층류**: Thwaites 방법 `θ² = 0.45/Re · ∫ue⁵ds / ue⁶`.
3. **천이**: `turbulent`는 정체점에서 코드 1% 지점 강제 천이(Fluent SST 완전 난류와 유사), `free`는 Michel 기준 `Reθ > 1.174(1 + 22400/Rex)·Rex^0.46` 또는 층류 박리(Thwaites λ < −0.09).
4. **난류**: Head 방법 + Ludwieg-Tillmann 마찰계수. 형상 계수 H ≥ 2.4이면 박리로 판정하고 그 지점 이후 비율을 `sep_fraction`으로 기록합니다.
5. **항력**: Squire-Young 식 `Cd = 2θ_TE · ue_TE^((H_TE + 5)/2)` (H는 2.5로 제한), 윗면·아랫면 합.

형상 지표

- 받음각 격자: 케이스 받음각 최솟값 ~ 최댓값 + `aoa_margin` + 4°(지면 모드 +2°)를 `aoa_step` 간격으로, 케이스 받음각도 포함.
- **추정 실속각**: 메인 흡입면(뒤집힌 날개는 아래쪽 면) 박리 비율이 `sep_limit`(10%)를 넘는 받음각(선형 보간).

- 실속각 이하 받음각 중 최대 Cl, 최대 L/D를 형상 점수로 씁니다.
- 지면 모드에서는 받음각마다 날개를 회전한 형상을 지면 거울상과 함께 다시 풉니다. 지면과 겹치는 받음각은 건너뜁니다.

9.2 선별 규칙

1. **선별 단위**: 케이스 세트(`case_set`, 2단/3단)와 `[results] split_by`(속도)의 조합마다 따로 선별합니다. 3단은 양력이 크지만 항력도 커서 L/D로는 2단보다 항상 불리하게 나오므로, 구성 간 비교는 CFD 결과로 합니다.
2. **계열 선별**: `protect` 변수를 뺀 나머지(익형, 플랩 코드, 플랩 각도 등)가 같은 형상들을 한 계열로 묶고, 계열 안 최고 점수로 비교합니다. 상위 `keep_top`개 계열과 최고 점수의 (1 − `keep_within`)배 이상인 계열만 남깁니다.
3. **보호 변수**: 슬롯 갭·오버랩은 합류 경계층 등 점성 효과가 커서 저충실도 예측이 부정확하고, 메인 코드는 길수록 Re가 커져 항상 유리하고, 지상고는 낮을수록 비점성 해석이 다운포스를 과대평가하므로, 이 값만으로는 제외하지 않습니다. 남은 계열 안에서는 보호 변수의 모든 값을 CFD로 돌립니다.
4. **받음각 선별** (`prune_aoa`): 유지된 형상에서 추정 실속각 + `aoa_margin`을 넘는 받음각은 제외하되, 넘는 첫 받음각 1개는 실속 이후 거동 확인용으로 남깁니다.
5. **스팟 체크**: 선별 단위마다 제외된 계열 중 최상위 `spot_check`개 계열의 최고 형상을, 추정 최대 Cl·최대 L/D 받음각에 가장 가까운 케이스 1~2개만 CFD로 돌립니다. `collect`에서 이 형상의 CFD 결과가 같은 조건의 유지된 형상 최저값보다 좋으면 "`keep_within`을 늘려 다시 screen"하라고 경고합니다.

9.3 병렬 처리와 성능

형상별로 모든 조건(속도)을 한 작업으로 묶어 `ProcessPoolExecutor`(spawn 방식)로 병렬 실행합니다. 같은 형상의 비점성 해는 작업 안에서 재사용합니다. 작업 프로세스마다 BLAS 스레드를 1개로 제한(`OPENBLAS/OMP/MKL_NUM_THREADS=1`)하는데, 그대로 두면 프로세스 수 × 코어 수만큼 스레드가 경합해 수십 배 느려지기 때문입니다(커밋 `eb0a5b3`: "30x faster parallel screening"). spawn 방식이라 환경변수가 numpy 로드 전에 적용됩니다.

9.4 지면 모드의 한계와 예제 설정

> **주의** 지면 근처에서는 비점성 해석이 날개 아래 흡입을 크게 과대평가합니다. README에 따르면 예제에서 추정 Cl max가 2단 6~8, 3단 13~18로 나오지만 실제는 대략 2~4입니다. 그 결과 박리가 일찍 판정돼 추정 실속각이 2~4°로 너무 낮게 나옵니다. 그래서 예제는 **받음각 선별을 끄고**(`prune_aoa = false`) 형상 계열 선별만 쓰며, 화면에도 절대값 대신 조건 안 최고 형상 대비 **상대 점수**(100 = 최고)만 표시합니다(커밋 `4728d4e`, `ff533df`).

일반적인 한계: 경계층과 비점성 유동의 상호작용, 슬롯의 합류 경계층, 실속 이후 유동은 고려하지 않습니다. 절대값(특히 고양력 형상의 Cl)은 CFD보다 높게 나오므로 **형상 간 상대 비교와 경향 확인용**으로만 써야 합니다. CFD가 끝난 뒤 `collect`가 그리는 `screening_vs_cfd_*.png`로 첫 배치에서 스크리닝 신뢰도를 확인하는 것이 권장 절차입니다.

9.5 예제 스크리닝 실행 결과

문서 작성 중 예제 프로젝트에 `aero-auto screen`을 그대로 실행한 결과입니다 (4코어 Linux 컨테이너).

1,056	3,464	867	459초
스크리닝 형상 단위 (264 형상 × 지상고 4, 형상 불가 0)	CFD 대상 케이스 (2단 1,732 + 3단 1,732, 스팟 체크 8 포함)	필요한 메쉬 (6,336개 중)	소요 시간 (4개 프로세스)

선별 단위	계열	유지된 계열 (순위순, 4개 속도 모두 동일)
2단 × 속도 4개	12 → 3	① 4412 메인 + flap1 150 mm 30° ② 2412 메인 + flap1 150 mm 30° ③ 4412 메인 + flap1 115 mm 30°

선별 단위	계열	유지된 계열 (순위순, 4개 속도 모두 동일)
3단 × 속도 4개	32 → 3	① 4412 + flap1 150 mm 25° + flap2 150 mm 45° ② 같은 플랩, 2412 메인 ③ 4412 + flap1 150 mm 15° + flap2 150 mm 45°

유지된 계열 안에서는 보호 변수(메인 코드 3 × 갭 2 × 지상고 4)의 모든 값이 남습니다. 계열 3개 × 24형상 × 받음각 6 × 속도 4 = 1,728케이스에 스팟 체크 4개를 더해 세트당 1,732케이스입니다. 각 조건의 1위 형상은 가장 낮은 지상고(30 mm)이고, 2단은 메인 코드가 길수록, 3단은 짧을수록 상대 점수가 높게 나왔습니다(2단: 450 mm 100 / 375 mm 97.4 / 300 mm 95.8, 3단: 300 mm 100 / 2412 메인 99.3 / 375 mm 98.3).

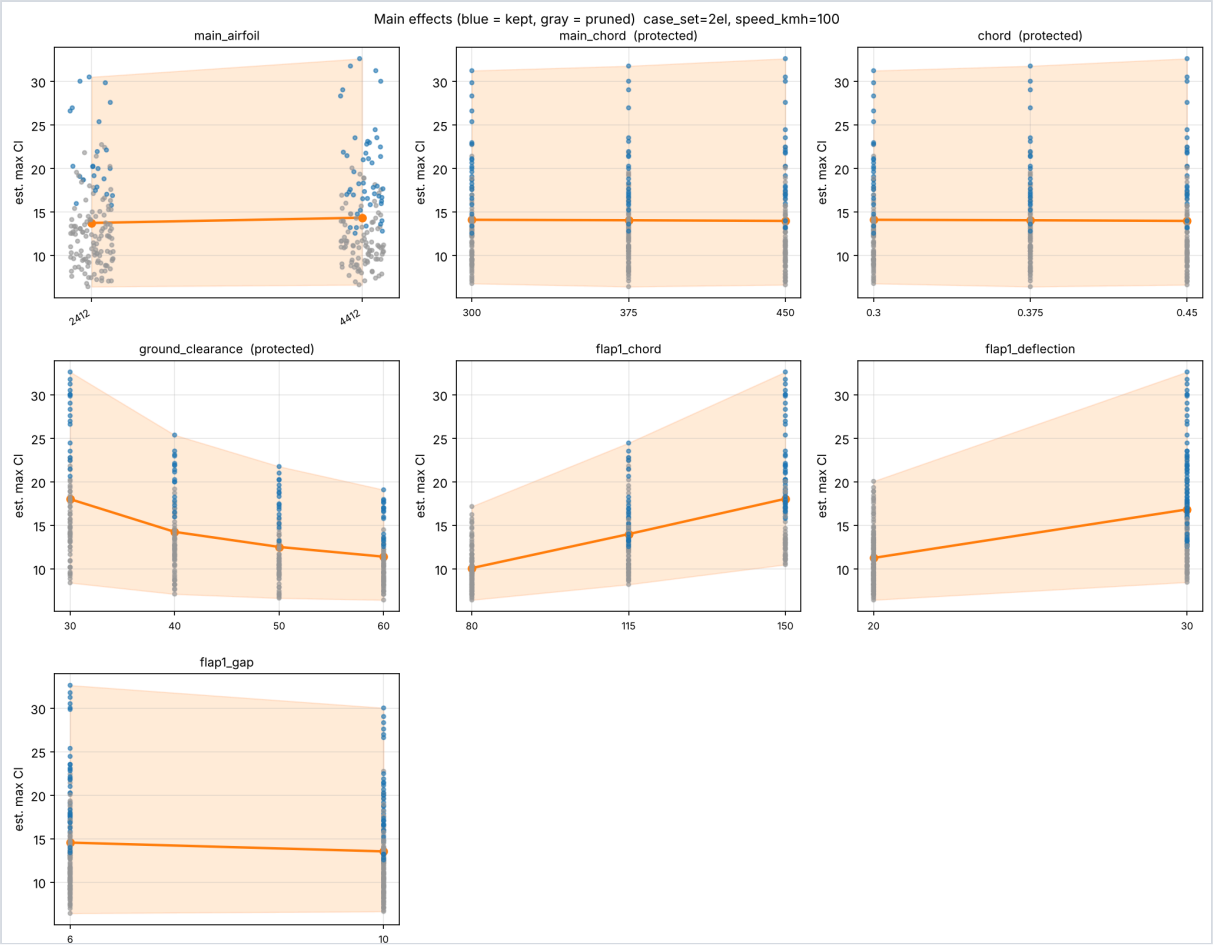


그림 9-2. `sensitivity_main_effects.png` (2단, 100 km/h). 플랩 코드·각도가 크고 지상고가 낮을수록 추정 다운포스가 큽니다. 원본 그림 아래쪽의 빈 `flap2_*` 패널은 잘라냈습니다(15.2절 참고).

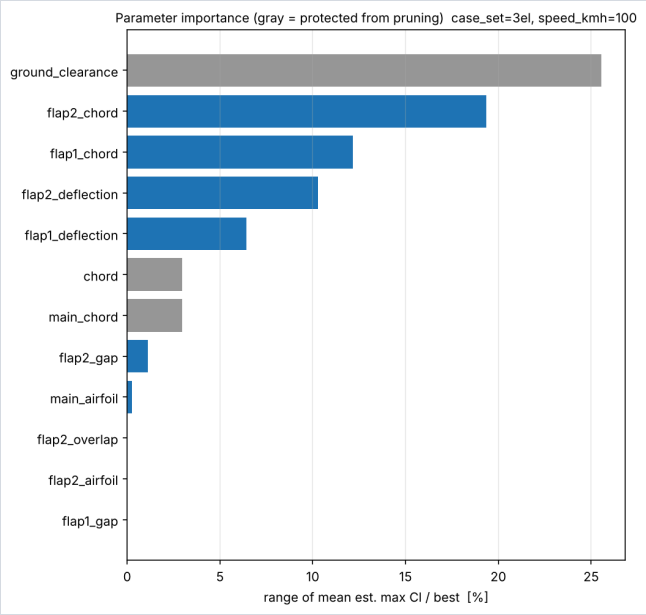


그림 9-3. `sensitivity_importance.png` (3단, 100 km/h). 지상고(보호) 다음으로 flap2 코드, flap1 코드, flap2 각도 순

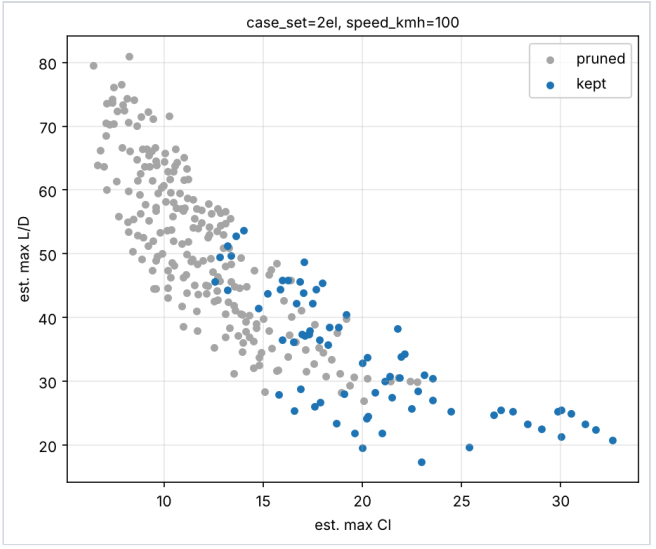


그림 9-4. `screening_scatter.png` (2단, 100 km/h). 파랑 = 유지, 회색 = 제외

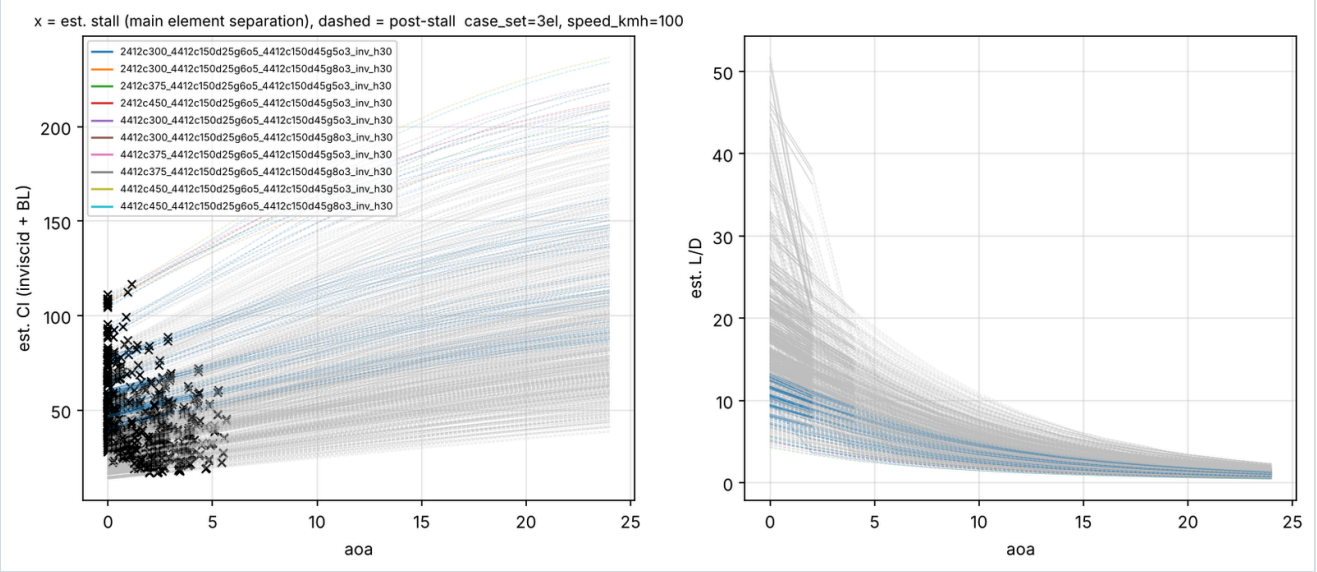


그림 9-5. `screening_polars.png` (3단, 100 km/h). × = 추정 실속각. 대부분 0~5°에 몰려 있어, 받음각 선별을 끈 예제 설정(9.4절)이 타당함을 보여줍니다.

> **관찰** 실제로 실행해 보니 추정 Cl max가 README의 설명(2단 6~8, 3단 13~18)보다 훨씬 크게 나왔습니다. 100 km/h 기준 중앙값(최대)은 2단 지상고 30 mm 16.5(32.6), 60 mm 10.8(19.1), 3단 30 mm 54.4(111.1), 60 mm 29.9(58.6)입니다. 또 지상고 30 mm에서는 추정 실속각 중앙값이 0°라서, 점수가 사실상 **받음각 0°의 비점성 Cl**로 정해집니다. 스팟 체크 케이스 8개도 모두 받음각 0°가 선택됐습니다. 상대 순위만 쓰는 설계라 절대값 자체는 문제가 아니지만, 저지상고에서 순위의 근거가 매우 좁다는 점을 감안해 첫 CFD 배치에서 `screening_vs_cfd_*.png` 로 순위 상관을 꼭 확인하는 것이 좋습니다.

9.6 스크리닝 산출물

파일 (runs/screening/)	내용
`<세트·속도>/sensitivity_main_effects.png`	변수 값별 점수 분포 (평균선, 최소~최대 범위, 형상별 점: 파랑 = 유지, 회색 = 제외)

파일 (runs/screening/)	내용
<세트·속도>/sensitivity_importance.png	변수 중요도: 변수 값에 따른 평균 점수 변화 폭 / 최고 점수 [%] (회색 = 보호 변수)
<세트·속도>/screening_polars.png	형상별 추정 Cl-α, L/D-α와 추정 실속각(×), 실속 이후는 점선
<세트·속도>/screening_scatter.png	형상별 추정 최대 Cl vs 최대 L/D
geometries.csv	형상·조건별 추정 실속각, 최대 Cl, 최대 L/D, 계열 순위, 유지 여부와 이유
cases.csv	케이스별 추정 Cl, Cd, L/D, 메인 Cl, 메인 박리 비율, 유지 여부와 이유, 스팟 체크 여부
selection.json	run · mesh 가 읽는 선별 결과: 전체 케이스, 유지/제외(이유), 스팟 체크, 지표, 설정

screen 을 다시 실행하면 이전 runs/screening/ 폴더를 지우고 새로 만듭니다. 스크리닝 이후에 추가된 케이스는 run 이 경고와 함께 스크리닝 없이 실행합니다.

10 Fluent 배치 실행

10.1 실행 방식: 베이스 케이스 + 메쉬 교체

해석 설정(모델, 물성, 경계조건, 리포트, 그래픽 객체, 뷰)은 베이스 케이스에 한 번 만들어 두고, 케이스마다 **베이스 케이스를 열고 → 메쉬만 교체하고 → 속도·기준값을 바꿔 → 계산**합니다. 2단과 3단은 경계 구성이 달라 `base_2el.cas.h5`와 `base_3el.cas.h5`를 따로 두고, 세트 상수 `base_case`로 저널이 골라 읽습니다.

10.2 예제 저널 템플릿 (journal.jou)

```
/file/read-case "${project_dir}/${base_case}"
/file/replace-mesh "${mesh_file}"
/mesh/check
/define/named-expressions/edit "speed" definition "${velocity:.6f} [m/s]" q
/report/reference-values/velocity ${velocity:.6f}
/report/reference-values/density 1.225
/report/reference-values/length ${chord}
/report/reference-values/area ${chord}
/solve/report-files/edit "coef-rfile" file-name "coef-rfile.out" q
/solve/initialize/hyb-initialization
/solve/iterate ${iterations}
/file/write-case-data "result.cas.h5"
/display/set/picture/driver png              ; 이하 이미지 저장 (graphics = true 필요)
/display/set/picture/x-resolution 1600
/display/set/picture/y-resolution 1000
/views/restore-view "near"
/display/objects/display "contour-pressure"
/display/save-picture "img_pressure.png"
/display/objects/display "contour-velocity"
/display/save-picture "img_velocity.png"
/views/restore-view "slot"
/display/objects/display "contour-velocity"
/display/save-picture "img_slot_velocity.png"
/exit yes
```

(주석 줄은 생략했습니다.) inlet 속도와 moving ground 속도가 모두 Named Expression `speed`를 참조하므로 한 줄로 함께 바뀝니다. 기준값을 케이스마다 갱신하지 않으면 계수가 베이스 케이스 속도로 무차원화되어 틀립니다. TUI 프롬프트는 Fluent 버전마다 다를 수 있어, 처음에는 GUI에서 저널을 녹화해 `/file/replace-mesh`, `/define/named-expressions/edit`, `/report/reference-values/...`, `/display/save-picture` 줄을 비교하는 것이 권장됩니다.

10.3 실행 명령

```
<fluent> 2ddp -t4 -i run.jou [-hidden | -gu -driver null | -g] [-wait] [extra_args...]
```

- Fluent 탐색: `AWP_ROOT<번호>` 환경변수를 번호 내림차순으로 보고 `fluent/ntbin/win64/fluent.exe`(Windows) 또는 `fluent/bin/fluent`가 있는 첫 번째를 씁니다. 없으면 PATH, 그래도 없으면 경로 예시와 함께 오류.
- `graphics = true`: Windows `-hidden`, Linux `-gu -driver null`(화면 없이 이미지 저장). 아니면 `-g`. Windows는 런처가 솔버 종료까지 기다리도록 `-wait`를 붙입니다.
- 작업 폴더는 케이스 폴더이며, 표준 출력·오류는 `solver.log`로 저장됩니다.

10.4 케이스 실행 절차와 상태

1. 상태가 `done`이면(그리고 `--force`가 아니면) 건너뜁니다.
2. 메쉬 파일이 없으면 `mesh_error.txt`의 이유와 함께 failed.

3. 이전 리포트 파일과 `*.png`를 지웁니다(Fluent가 덮어쓰기 확인을 물으면 배치가 멈추기 때문).
4. 저널을 치환해 `run.jou`, 변수를 `case.json`에 쓰고, 상태를 running으로 기록한 뒤 실행합니다.
5. 판정: 종료 코드 ≠ 0 → failed / 로그에 오류 패턴 → failed / 시간 초과 → 프로세스 트리 강제 종료 후 failed / 그 외 done.

상태	의미
`pending`	`status.json` 없음 (아직 실행 안 함)
`running`	실행 중 (명령, 시작 시각 기록)
`done`	정상 완료 (종료 코드, 소요 시간 기록)
`failed`	실행 실패, 로그 오류, 시간 초과, 메쉬 실패 (이유 기록)
`interrupted`	Ctrl+C로 중단
`screened`	스크리닝에서 제외 (이유 기록). 이미 done인 케이스는 바꾸지 않음

프로세스 트리 종료: Fluent는 런처 아래로 cortex/MPI 프로세스를 띄우므로 트리 전체를 종료합니다. Windows는 `taskkill /F /T`, Linux는 새 세션으로 띄운 뒤 `killpg(SIGKILL)`.

병렬 실행: `max_parallel`(또는 `-j`) > 1이면 스레드 풀로 여러 Fluent를 동시에 띄웁니다. 메쉬는 실행 전에 순차로 모두 만듭니다. 동시 실행 수는 Fluent 라이선스 수와 코어 수(`processes` × 동시 실행 수)를 고려해 정합니다.

10.5 Fluent 베이스 케이스 설정 요약

`docs/FLUENT_BASE_CASE.md`(Fluent 2026 R1 기준, 화면 단위 가이드)의 핵심을 표로 정리했습니다. 상세 절차는 부록 A에 있습니다.

항목	설정
Launcher	2D, Double Precision, Solver Processes 4, Working Directory = 예제 폴더
General	Pressure-Based, Steady, Planar
Viscous	Transition SST (Re 23만~86만, 층류 박리 기포·천이 포착) 또는 k-ω SST(완전 난류, 이때 스크리닝 `transition = "turbulent"` 권장)
Materials (air)	Density constant 1.225 kg/m³, Viscosity 1.7894e-05 kg/(m·s) — 마하 0.1 미만이라 비압축
Named Expression	`speed` = `27.78 [m/s]` (콘솔: `/define/named-expressions/add "speed" definition "27.78 [m/s]" q`)
`inlet`	Velocity Magnitude = `speed`, Normal to Boundary, 난류 강도 0.5%, 점성비 10
`outlet`	Gauge Pressure 0, backflow 난류 inlet과 동일
`top`	symmetry
`ground`	**Moving Wall**, Absolute, Translational, Speed = `speed`, Direction (1, 0), No Slip
`main`, `flap1`(, `flap2`)	Stationary Wall, No Slip
Reference Values	Compute from inlet, Area = Length = 메인 코드 (0.375), Density 1.225
Methods	Coupled, Pseudo Time, Least Squares, Pressure 2nd order, 나머지 2nd order upwind
Report Definitions	Force Coefficient `cx` (1,0), `cy` (0,1) — 날개 wall 전체 / `cx-main`, `cy-main`, `cx-flap1`, `cy-flap1`(, `-flap2`) — 요소별. **ground는 Zones에 넣지 않음**
Report File	`coef-rfile` → `coef-rfile.out`, 매 반복
Graphics / Views	Contour `contour-pressure`(Cp), `contour-velocity`(속도 크기), 뷰 `near`(날개 전체), `slot`(슬롯 확대)
저장	`base_2el.cas.h5` (Case만). 3단은 Replace Mesh 후 `flap2` 리포트 추가해 `base_3el.cas.h5`

지면이 없는(자유류) 프로젝트: 경계가 `farfield` 하나입니다. Density = ideal-gas, Named Expression `flow_angle`, `mach`를 만들고, farfield(pressure-far-field)에 Mach = `mach`, 방향 = (cos, sin)(`flow_angle`)을 넣고, 저널은 `speed` 대신 `flow_angle`과 `mach`를 바꾸도록 씁니다(git 기록의 이전 `journal.jou` 참고).

11 결과 처리

11.1 리포트 파일 파싱

Fluent report-file 형식(첫 줄 이름, 따옴표로 둘러싼 헤더 줄, 숫자 행)을 읽습니다. 헤더가 2개 이상인 따옴표 줄을 헤더로, 헤더 수와 열 수가 같은 숫자 행을 데이터로 씁니다. `Iteration`, `Time Step`, `flow-time` 열은 값에서 제외하고, 나머지 열은 **마지막 `average_last` 개 반복의 평균**을 결과값으로 씁니다(진동하며 수렴하는 경우 대비). 여러 리포트 파일에 같은 이름이 있으면 `파일이름.열이름`으로 구분합니다.

11.2 물체축 → 풍축 변환

Fluent에는 받음각과 무관한 물체축 계수 cx(x 방향), cy(y 방향)만 리포트로 걸어 두고, 도구가 유입 각도 θ(`flow_angle`, 없으면 `aoa`)로 회전해 양력·항력 계수를 만듭니다. 받음각이 바뀌어도 Fluent 쪽 force vector를 매번 바꿀 필요가 없습니다.

```
cl = s · (cy·cosθ − cx·sinθ), cd = cx·cosθ + cy·sinθ, s = −1 (downforce) / +1 l/d = cl / cd
```

`cx`로 시작하는 열마다 같은 접미사의 `cy` 열을 짝지어 변환하므로, 요소별(`cx-flap1` → `cl-flap1`, `cd-flap1`)도 자동입니다. 지면 모드는 θ = 0이라 cl = −cy(다운포스), cd = cx입니다.

11.3 단위 폭당 힘

$$q = \tfrac{1}{2}\,\rho\,V^2,\quad F_{lift(or\ downforce)} = q \cdot c \cdot cl\ [N/m],\quad F_{drag} = q \cdot c \cdot cd\ [N/m]$$

케이스 변수 `velocity`[m/s]와 `chord`[m]가 있으면 계산합니다(ρ는 변수 `density`, 없으면 1.225). 계수는 각자의 메인 코드로 무차원화되므로, 메인 코드나 구성(2단/3단)이 다른 형상끼리 실제 힘을 비교할 때는 이 열(`downforce_N_per_m`, `drag_N_per_m`)을 봐야 합니다.

11.4 형상 순위 (ranking.csv)

- 형상 구분 변수: 케이스마다 값이 다른 변수 중 받음각과 파생 변수를 뺀 것 (`split_by` 변수는 맨 뒤로).
- 완료(done)되고 cl이 있는 케이스만 사용. 형상별로 최대 Cl과 그때 받음각, 최대 L/D와 그때 받음각·Cl·Cd, 받음각 개수(`n_aoa`), 형상 이름을 기록합니다.
- `objective`(최대 L/D 또는 최대 Cl)로 내림차순 정렬하고, `split_by`가 있으면 조건(속도)마다 1위부터 순위를 따로 매깁니다. 2단·3단은 함께 비교합니다.
- 화면에는 조건별 상위 3개(조건이 없으면 상위 5개)를 출력합니다.

11.5 출력 파일

파일	내용
`runs/summary.csv`	케이스별 모든 변수, 메쉬·형상 이름, 상태, 소요 시간(분), 반복 수, 리포트 평균값, cl/cd/l/d(요소별 포함), 단위 폭당 힘. UTF-8 BOM으로 저장해 엑셀에서 한글이 깨지지 않음
`runs/ranking.csv`	형상별 순위 (11.4절)
`runs/plots/<조건>/`	`cl_alpha.png`, `cd_alpha.png`, `ld_alpha.png`, `polar.png` (예: `speed_kmh=100/`)
`runs/plots/by_speed_kmh.png`	상위 형상의 최대 L/D(또는 Cl)가 속도에 따라 변하는 경향
`runs/plots/screening_vs_cfd_*.png`	스크리닝 추정 vs CFD (상관계수 r 표시)

파일	내용
`runs/<그룹>/<케이스>/`	`run.jou`, `case.json`, `status.json`, `solver.log`, `coef-rfile.out`, `convergence.png`, `img_*.png`, `result.cas.h5`
`runs/_meshes/<형상>/`	`mesh.msh`, `mesh_info.json`, `geometry.png`, `mesh.png`, `mesh_slot1.png` (, `mesh_slot2.png`), `mesh_le.png`, 실패 시 `mesh_error.txt`
`runs/screening/`	9.6절

12 예제 프로젝트: flap_optimization

12.1 케이스 구성

구분	변수와 값	조합
공통 상수	`downforce = true`, `ground_ref = "lowest"`, `iterations = 2000`	
공통 스윕	`main_airfoil` 2412, 4412 · `main_chord` 300, 375, 450 mm · `ground_clearance` 30, 40, 50, 60 mm · `aoa` 0, 4, 8, 12, 16, 20° · `speed_kmh` 40, 60, 80, 100	576
2단 세트 `2el`	상수: `base_2el.cas.h5`, flap1 4412, overlap 5 mm 스윕: `flap1_chord` 80, 115, 150 · `flap1_deflection` 20, 30 · `flap1_gap` 6, 10	× 12 = 6,912
3단 세트 `3el`	상수: `base_3el.cas.h5`, flap1 4412 gap 6 overlap 5, flap2 4412 overlap 3 스윕: `flap1_chord` 80, 150 · `flap1_deflection` 15, 25 · `flap2_chord` 80, 150 · `flap2_deflection` 35, 45 · `flap2_gap` 5, 8	× 32 = 18,432
합계	형상 264개 (2단 72 + 3단 192) × 지상고 4 × 받음각 6 × 속도 4	**25,344**

그룹(분류 폴더) 패턴은 `{case_set}_{main_airfoil}c{main_chord}_h{ground_clearance}` 이고, 케이스 이름은 세트마다 다른 패턴을 씁니다(예: `2el_4412c375_f115d20g6_h30_v100_a4`).

12.2 속도와 레이놀즈수

```
chord = main_chord / 1000 [m], velocity = speed_kmh / 3.6 [m/s], mach = velocity / 340.29, reynolds = 1.225 · velocity · chord / 1.7894e-5
```

속도	유속	마하수	Re (메인 300 mm)	Re (375 mm)	Re (450 mm)
40 km/h	11.1 m/s	0.033	228,000	285,000	342,000
60 km/h	16.7 m/s	0.049	342,000	428,000	513,000
80 km/h	22.2 m/s	0.065	456,000	570,000	685,000
100 km/h	27.8 m/s	0.082	570,000	713,000	856,000

Re는 입력값이 아니라 속도와 코드 길이로 정해지는 결과입니다. 메쉬는 형상(지면 모드는 지상고·받음각 포함)별로 하나만 만들고 속도와는 공유하며, 첫 셀 높이는 그 메쉬를 쓰는 케이스 중 가장 높은 Re 기준이라 모든 속도에서 y+ ≤ 1입니다.

12.3 예제 설정의 주요 선택

설정	이유 (설정 파일 주석 기준)
`objective = "max_cl"`	형상 순위를 최대 다운포스로. 효율이 중요하면 `max_ld`
`split_by = ["speed_kmh"]`	속도별로 순위·그래프·스크리닝 선별을 따로 수행 (특정 속도에서만 좋은 형상도 그 속도에서는 유지)
`average_last = 100`	마지막 100 반복 평균
`transition = "free"`	Re 수십만은 천이 영향이 커서 자연 천이로 추정 (Fluent는 Transition SST 권장)
`aoa_step = 2.0`	지면 모드는 받음각마다 형상을 다시 풀어서 간격을 넓힘
`prune_aoa = false`	지면 모드에서 실속각이 너무 낮게 추정됨 (9.4절)
`graphics = true`, `timeout_minutes = 60`	컨투어 이미지 저장, 케이스당 최대 1시간

12.4 파일럿 확인 순서

1. `sample_meshes.zip`의 메쉬 2개를 Fluent에서 열어 Mesh Check를 확인합니다 (2단·3단, 둘 다 지상고 30 mm, 받음각 4°).
2. 베이스 케이스를 만든 뒤 `File > Replace Mesh`로 같은 구성의 다른 메쉬로 바꿔도 경계조건, 리포트, 뷰가 유지되는지 확인합니다.
3. 케이스 하나씩 배치로 실행합니다:

   ```
   aero-auto run --all --only "2el_4412c375_f115d20g6_h30_v100_a4"
   aero-auto run --all --only "3el_4412c375_f80d25_f80d35g8_h30_v100_a4"
   ```

4. `solver.log`에 오류가 없는지, `coef-rfile.out`, `img_*.png`, `result.cas.h5`가 생겼는지, `aero-auto status`의 케이스당 소요 시간을 확인합니다.

13 테스트

실제 Fluent 대신 가짜 솔버(`tests/fake_solver.py`)를 `kind = "custom"`으로 실행해 전체 흐름을 검증합니다. 가짜 솔버는 저널에서 `AOA=`, `GAP=`, `MESH=`를 읽어 Fluent 형식 리포트 파일을 씁니다. 갭 0.02에서 최대가 되는 가상의 양력을 만들어 순위가 맞는지 확인할 수 있고, `AOA ≥ 99`면 로그에 발산 오류를, `AOA ≤ −99`면 종료 코드 3을 내서 실패 판정을 시험합니다.

문서 작성 시 `pytest`를 실행한 결과 **43개 전부 통과**(약 41초)했습니다.

파일	테스트 (검증 내용)
`test_workflow.py` (6)	케이스 그룹·파생 변수 / 이름 중복 거부 / 템플릿 미정의 변수 보고 / 리포트 파싱 / 실행→상태→수집(실패 판정 포함) / 시간 초과 시 failed
`test_geometry_mesh.py` (8)	NACA 정규화 / .dat 읽기 / 갭 정확도(여러 값) / 불가능한 갭 거부 / .msh 면 방향 규칙 / 경계 미지정 거부 / 메쉬→해석→cl/cd→순위→그래프 전체 흐름 / 메쉬 실패 시 케이스 failed
`test_multi_element.py` (6)	3단 형상 mm 단위 배치 / 메쉬 이름에 모든 요소 포함 / 3단 메쉬 / 3단 패널법 / 2단·3단 케이스 세트 / 세트별 선별
`test_screening.py` (9)	패널법 이론값 / 받음각에 따른 박리 증가 / 플랩의 양력 증가 / 선별과 run 반영 / 스팟 체크 경고 / 속도별 스크리닝 / 조건별 순위 / 코드 스윕 메쉬와 힘 / 단위 폭당 힘
`test_downforce.py` (5)	뒤집힌 형상 = 거울상 / 뒤집힌 메쉬 / 다운포스 부호 / 스크리닝 다운포스 = 똑바른 양력 / flow_angle 변수
`test_ground.py` (7)	최하단 기준 배치와 앞전 중심 회전 / 앞전 기준 지상고 / 메쉬 이름의 지상고·받음각 / 지면 메쉬 경계 / 지면 효과 경향(낮을수록 다운포스 증가) / 지면 모드 풍축 변환 / 지면 케이스와 스크리닝

파라미터화된 테스트(갭 정확도)가 있어 함수 개수(41)와 실행 개수(43)가 다릅니다.

14 개발 이력

구현 브랜치의 커밋 기록입니다 (오래된 순). 2026-10-29~30 이틀 동안 기본 배치 도구에서 시작해 형상·메쉬 생성, 스크리닝, 속도 분석, 다요소, 다운포스, 지면 효과 순으로 확장됐습니다.

커밋	날짜	내용
`c93c9ed`	10-29	초기 커밋 (README 한 줄, .gitignore)
`6f9f27c`	10-29	ANSYS Fluent 케이스 스윕 배치 자동화 도구 추가 (케이스 전개, 저널 치환, 실행, 상태, 리포트 수집)
`83ee187`	10-30	2D 메인 + 플랩 형상 생성, gmsh 메쉬, 결과 후처리 추가
`f1e148e`	10-30	CFD 전 스크리닝과 민감도 그래프로 가망 없는 케이스 제외
`23aa9d1`	10-30	차량 속도별 분석: 속도 스윕, 마하/Re 파생, 속도별 선별
`da9237d`	10-30	코드 길이를 설계 변수로 허용
`83c1fa5`	10-30	2단·3단 날개, 요소별 코드 길이(mm) 지원
`f4e49cd`	10-30	다운포스용 뒤집힌 날개로 전환
`4bb80c2`	10-30	지면 효과 추가: 움직이는 바닥, 지상고 스윕, 메인 앞전 기준 받음각 회전
`7c24269`	10-30	지면 효과 샘플 메쉬 포함, 파일럿 절차 갱신
`81e2f23`	10-30	예제와 오류 안내에 Fluent 2026 R1(v261) 경로 사용
`eb0a5b3`	10-30	스크리닝 작업 프로세스의 BLAS 스레드 제한 (병렬 스크리닝 30배 가속)
`ff533df`	10-30	지면 효과 예제에서 받음각 선별 끔
`4728d4e`	10-30	지면 모드 스크리닝에서 절대 추정값 대신 상대 점수 표시
`7671f4a`	10-30	예제 형상 순위를 최대 다운포스 기준으로
`0d0cc75`	10-30	지면 효과 설정용 Fluent 베이스 케이스 단계별 가이드 추가
`96db077`	10-30	Fluent 가이드: speed Named Expression을 콘솔 또는 Input Parameters로 만드는 방법

15 한계, 주의사항, 검토 메모

15.1 방법론의 한계 (설계상 의도된 것)

- **스크리닝은 상대 비교용**: 패널법 + 적분 경계층은 경계층-비점성 상호작용, 슬롯 합류 경계층, 실속 이후 유동을 고려하지 않습니다. 지면 근처에서는 흡입을 몇 배 과대평가합니다. 그래서 슬롯 변수·메인 코드·지상고는 보호 변수로 두고, 스팟 체크와 스크리닝-CFD 상관 그래프로 사후 검증하도록 설계돼 있습니다.
- **2D 해석**: 단위 폭당 힘과 2D 계수입니다. 날개 끝 와류 등 3D 효과는 없습니다.
- **지면 모드 메쉬 수**: 받음각을 형상 회전으로 반영하므로 받음각마다 메쉬가 필요합니다(예제 전체 6,336개). 스크리닝으로 줄인 뒤 메쉬를 만드는 순서가 중요합니다.
- **Fluent 저널 호환성**: TUI 명령 형식은 버전마다 다를 수 있어 첫 실행 전에 녹화해 비교해야 합니다.
- **플랩 수**: 코드상 최대 3개(`MAX_FLAPS = 3`), 문서·예제는 1~2개를 다룹니다.

15.2 코드 검토 중 발견한 사항

문서화를 위해 전체 코드를 읽으면서 확인한 점입니다. 동작을 막는 버그는 아니지만 알아 두면 좋은 내용입니다.

항목	내용
README와 코드의 기본값 차이	`[screening] protect` 기본값: README 표는 4개, 코드(`screening.py`)는 `ground_clearance` 를 포함한 5개. 예제는 5개를 명시하므로 동작에는 영향 없음
`[solver] mode` 기본값 `"3ddp"`	도구는 2D 메쉬만 만들지만 기본값은 3ddp입니다. `mode` 를 빠뜨리면 Fluent가 2D 메쉬를 3D 솔버로 읽으려다 실패합니다. 예제는 `"2ddp"` 를 명시
`run --only` 후 수집 범위	`run` 은 끝나면 `--only` 와 무관하게 **전체** 케이스로 `collect` 를 실행합니다(케이스별 수렴 그래프 포함). 케이스가 많으면 매 실행 끝에 수집 시간이 걸릴 수 있음
로그 오류 패턴	`Error:` 등은 대소문자 무시 부분 일치입니다. Fluent가 무해한 경고 문구에 "error:"를 포함하면 정상 케이스도 failed가 될 수 있으므로, 오탐이 있으면 `[results] error_patterns` 를 조정
세트별 변수 목록	스크리닝의 "형상 변수 중 값이 변하는 것"(`varying`)을 전체 케이스 기준으로 계산합니다. 그래서 2단 조건의 민감도 그래프에 값이 없는 `flap2_*` 패널이 빈 채로 그려지고, 2단 계열 이름에 `flap2_airfoil=None` 같은 항목이 붙습니다. 선별 결과에는 영향 없음(표시 문제)
지면 모드 추정값 크기	README의 추정 Cl max 범위(2단 6~8, 3단 13~18)가 현재 코드의 실제 출력(9.5절: 2단 최대 33, 3단 최대 111)과 다릅니다. 지상고 30 mm에서 추정 실속각이 0°로 나와 점수가 받음각 0°의 값으로 정해지는 점도 README에 없는 내용
main 브랜치	구현이 아직 main에 병합되지 않았습니다(1.3절)

A 부록: Fluent 베이스 케이스 작성 절차

`docs/FLUENT_BASE_CASE.md`의 순서를 그대로 옮겼습니다. 메뉴 이름은 Fluent 버전에 따라 조금씩 다를 수 있습니다.

1. **준비**: `sample_meshes.zip`을 예제 폴더에 풉니다. Fluent Launcher에서 2D, Double Precision, Solver Processes 4, Working Directory를 예제 폴더로 설정합니다.
2. **메쉬 읽기와 확인**: `File > Read > Mesh`로 2단 `mesh.msh`. `Mesh > Check`에 ERROR가 없어야 하며, Domain Extents가 대략 x −1.9~4.5 m, 높이 약 2.25 m면 정상. `Mesh > Quality`의 Minimum Orthogonal Quality ≥ 0.1이면 양호. 경계 타입이 8.2절 표와 다르면 수정.
3. **General**: Pressure-Based, Steady, Planar.
4. **Viscous**: Transition SST (불안정하면 k-ω SST).
5. **Materials**: air, density constant 1.225, viscosity constant 1.7894e-05. Cell Zone `fluid`의 재료가 air인지 확인.
6. **Named Expression** `speed`: 콘솔 `/define/named-expressions/add "speed" definition "27.78 [m/s]" q` (확인: `/define/named-expressions/list`) 또는 `Parameters & Customization > Parameters > Input Parameters`에서 New.
7. **Boundary Conditions**: inlet(Magnitude, Normal to Boundary, `speed`, 0.5%, 10), outlet(0 Pa, backflow 0.5%, 10), top(symmetry), ground(Moving Wall, Absolute, Translational, `speed`, (1, 0), No Slip), main/flap1(stationary).
8. **Reference Values**: Compute from inlet, Area 0.375, Length 0.375, Density 1.225, Velocity 27.78, Reference Zone fluid.
9. **Methods**: Coupled, Pseudo Time Method, Least Squares Cell Based, Pressure Second Order, 나머지 Second Order Upwind. Controls 기본값.
10. **Report Definitions**: Force Report(Force Coefficient) 6개 — `cx`(1,0; main+flap1), `cy`(0,1; main+flap1), `cx-main`, `cy-main`, `cx-flap1`, `cy-flap1`. 이름은 정확히 같아야 하고, ground는 넣지 않습니다. 뒤집힌 날개라 cy는 음수가 정상.
11. **Report File**: `coef-rfile`에 6개 리포트, 파일 이름 `coef-rfile.out`, 매 반복.
12. **그래픽**: Contour `contour-pressure`(Pressure Coefficient, Filled), `contour-velocity`(Velocity Magnitude, Filled). 뷰 `near`(날개 전체 + 지면), `slot`(슬롯 확대) 저장.
13. **동작 확인**(권장): Hybrid 초기화, 20 반복 → 오류 없이 `coef-rfile.out`이 생기고 cy가 음수인지 확인.
14. **저장**: `File > Write > Case` → `base_2el.cas.h5` (Case만).
15. **3단**: `base_2el`이 열린 상태에서 Replace Mesh로 3단 메쉬 → `flap2`가 wall인지 확인 → `cx`, `cy` Zones에 flap2 추가, `cx-flap2`, `cy-flap2` 생성 → Report File에 추가 → 뷰 확인 → `base_3el.cas.h5`. 오류가 나면 3단 메쉬로 처음부터.
16. **저널 녹화**(권장): `File > Write > Start Journal` → Replace Mesh, speed 수정, Reference Velocity 수정, Save Picture 수행 → Stop Journal. 녹화된 명령 형식에 맞춰 `journal.jou`를 고칩니다.
17. **파일럿 실행**: 12.4절.

B 부록: 용어

용어	뜻
코드 (chord), c	익형 앞전~뒷전 직선 길이. 이 문서에서 길이 단위 "c"는 메인 코드 배수
받음각 (aoa, α)	유동과 코드선 사이 각도. 이 도구에서는 다운포스(또는 양력)가 커지는 방향이 +
슬롯 플랩	메인 뒷전 뒤에 틈(슬롯)을 두고 배치한 보조 날개. 슬롯 유동이 플랩 경계층에 에너지를 줘 박리를 늦춤
갭 / 오버랩	슬롯의 최소 틈 / 앞 요소 뒷전과 플랩 앞쪽 끝의 수평 겹침 (7.1절)
지상고	지면과 날개 사이 거리 (기본: 날개 최하단 기준)
Cl, Cd, L/D	양력(다운포스) 계수, 항력 계수, 양항비 (다운포스 날개는 다운포스/항력)
cx, cy	물체축(전역 x, y 방향) 힘 계수. 도구가 cl, cd로 변환
Re, 마하수	레이놀즈수 ρVc/μ, 유속/음속
y+	벽면 첫 셀 높이의 무차원 값. SST 계열 모델은 y+ ≈ 1 권장
패널법	물체 표면을 소스·와류 패널로 근사해 비점성 유동을 푸는 방법
적분 경계층	경계층 두께 지표(운동량 두께 θ, 형상 계수 H)의 적분 방정식으로 경계층 성장과 박리를 추정하는 방법
계열 (family)	스크리닝에서 보호 변수를 뺀 나머지 형상 변수가 같은 형상들의 묶음
스팟 체크	제외된 계열의 대표 형상을 소수 케이스만 CFD로 돌려 스크리닝을 검증하는 장치
베이스 케이스	해석 설정을 담은 Fluent 케이스 파일. 케이스마다 메쉬만 교체해 사용
Moving wall	벽이 접선 방향으로 움직이는 경계조건. 차량 기준 좌표에서 도로를 표현